

Relatório de Inspeção Técnica — Segunda Edição

Relatório de Evolução da Plataforma Capacita
Portos (abril–maio de 2026)

Autoria:	Equipe de Análise Técnica
Emissão:	19 de maio de 2026
Plataforma:	Capacita Portos
Referência:	Relatório de Inspeção Técnica Exaustivo (1ª edição)
Norma:	ISO/IEC 25010:2015

Sumário

Sumário

Relatório de Evolução da Plataforma de e-Learning Gamificada "Capacita Portos"

1.1 Objetivo do Documento

1.2 Relação com o Primeiro Relatório

1.3 Escopo e Metodologia

1.4 Período Coberto e Linha do Tempo

1.5 Quadro-Resumo das Alterações

2.1 Classificação das Mudanças por Categoria

2.2 Mapa de Impacto sobre os Subsistemas

2.3 Arquivos Novos versus Arquivos Modificados

3.1 Contexto e Motivação

3.2 O Gerador generate_sql.py

3.3 Script de Atualização de E-mails — sql_trocar_emails_2026_03_30.sql

3.4 Script de Inserção de Novos Alunos — sql_inserir_novos_alunos_2026_03_30.sql

3.5 Script de Enforcement de Troca de Senha — sql_forcar_troca_senha_novos_alunos_2026_03_31.sql

3.6 Análise de Segurança e Privacidade da Entrega

3.7 Recomendações Específicas da Entrega 1

4.1 Motivação e Modelo Conceitual

4.2 Camada de Domínio — src/lib/accessLevels.js

4.3 Camada de Persistência — supabase/migration_access_levels.sql

4.3.1 Etapa 1 — Criação do Tipo Enumerado

4.3.2 Etapa 2 — Adição da Coluna

4.3.3 Etapa 3 — Retrocompatibilidade de Dados

4.3.4 Etapa 4 — Função de Leitura do Próprio Nível

4.3.5 Etapa 5 — Função de Escrita (Administrativa)

4.3.6 Etapa 6 — Atualização da Função get_platform_users

4.4 Camada de Estado — src/contexts/AuthContext.jsx

4.5 Camada de Apresentação — src/pages/DisciplineDetail.jsx

4.6 Camada de Administração — src/pages/admin/AdminUsers.jsx

4.7 Análise de Padrões de Projeto da Entrega 2

4.8 Análise de Segurança da Entrega 2

4.9 Recomendações Específicas da Entrega 2

5.1 Motivação

5.2 Nova Dependência — xlsx

5.3 A Função exportStudentsToExcel

5.3.1 Fase 1 — Seleção do Conjunto de Dados

5.3.2 Fase 2 — Construção da Planilha "Resumo"

5.3.3 Fase 3 — Construção da Planilha "Detalhado"

5.3.4 Fase 4 — Montagem e Escrita do Arquivo

5.4 Avaliação da Qualidade da Implementação

5.5 Interface e Estilos

5.6 Recomendações Específicas da Entrega 3

6.1 Contexto e Motivação

6.2 As Políticas de Row Level Security

6.3 Descoberta Relevante — Fechamento de uma Lacuna Funcional

6.4 Análise de Segurança da Entrega 4

6.5 Recomendações Específicas da Entrega 4

7.1 Contexto e Motivação

7.2 As Funções addQuizOption e removeQuizOption

7.3 Análise da Lógica de Reindexação da Resposta Correta

7.4 Interface e Restrições

7.5 Avaliação da Qualidade da Implementação

7.6 Recomendações Específicas da Entrega 5

8.1 Impacto sobre a Arquitetura

8.2 Impacto sobre a Segurança

- 8.3 Impacto sobre a Manutenibilidade e a Complexidade**
- 8.4 Impacto sobre as Dependências**
- 8.5 Impacto sobre a Testabilidade**
- 8.6 Situação dos Pontos Críticos Herdados da Primeira Edição**
- 8.7 Reavaliação dos Índices de Qualidade ISO/IEC 25010**
- 9.1 Resumo Executivo**
- 9.2 Pontos Fortes do Período**
- 9.3 Pontos de Atenção do Período**
- 9.4 Recomendações Prioritárias para o Próximo Ciclo**
- 9.5 Matriz de Riscos**
- 9.6 Considerações Finais**
- 10.1 Apresentação Geral e Justificativa Estratégica**
- 10.2 Público-Alvo: O Servidor e o Quadro Técnico Portuário**
- 10.3 Diferenciação em Relação à Plataforma Atual**
- 10.4 Arquitetura Prevista (Espelhada da Plataforma Atual)**
- 10.5 Catálogo Detalhado de Funcionalidades**
 - 10.5.1 Autenticação Institucional
 - 10.5.2 Trilha de Disciplinas Sequenciais
 - 10.5.3 Aulas em Vídeo
 - 10.5.4 Materiais Complementares
 - 10.5.5 Quiz por Aula
 - 10.5.6 Quiz Final da Disciplina
 - 10.5.7 Sistema de Badges e Gamificação
 - 10.5.8 Ranking e Discipline Ranking
 - 10.5.9 Fórum
 - 10.5.10 Minhas Dúvidas e Sistema de Monitoria
 - 10.5.11 Chat com Inteligência Artificial (Google Gemini)
 - 10.5.12 Painel do Monitor
 - 10.5.13 Painel Administrativo
 - 10.5.14 Sistema de Níveis de Acesso
 - 10.5.15 Exportação de Relatórios para Excel

- 10.5.16 Moderação Administrativa do Fórum
- 10.5.17 Recuperação e Redefinição de Senha
- 10.5.18 Enforcement de Troca de Senha no Primeiro Acesso
- 10.5.19 Notificações e Badge de Dúvidas
- 10.5.20 Layout Responsivo, Mobile e Sidebar
- 10.5.21 Dashboard do Aluno
- 10.5.22 Sinalização de Disciplinas Concluídas
- 10.5.23 Política de Acesso e Segurança da Sessão

10.6 Conteúdo Pedagógico Especializado — Eixos Temáticos

10.7 Integração com Sistemas Internos — Possibilidades Futuras

10.8 Segurança Reforçada — Quadro Mínimo

10.9 Conformidade — LGPD e Princípios da Administração Pública

10.10 Cronograma e Marcos — Visão Indicativa

10.11 Riscos e Mitigações Específicas

10.12 Sinergia com o Projeto Atual — Estratégia de Código Compartilhado

10.13 Conclusão do Capítulo

11.1 Visão Geral do Stack Tecnológico

11.2 Linguagens de Programação e Marcação

- 11.2.1 JavaScript (ECMAScript 2020+)
- 11.2.2 JSX (JavaScript XML)
- 11.2.3 HTML5 e CSS3
- 11.2.4 SQL (PostgreSQL Dialect)
- 11.2.5 Python 3
- 11.2.6 Bash / Shell

11.3 Framework Principal: React 19

- 11.3.1 React (react) — versão ^19.2.4
- 11.3.2 React DOM (react-dom) — versão ^19.2.4
- 11.3.3 React Router DOM (react-router-dom) — versão ^7.13.0
- 11.3.4 React Icons (react-icons) — versão ^5.5.0

11.4 Ferramentas de Build, Bundling e Desenvolvimento

- 11.4.1 Vite (vite) — versão ^7.3.1
- 11.4.2 Plugin Vite para React (@vitejs/plugin-react) — versão ^5.1.4

11.5 Qualidade de Código e Linting

11.5.1 ESLint (eslint) — versão ^9.39.2

11.5.2 ESLint JS (@eslint/js) — versão ^9.39.2

11.5.3 ESLint Plugin React Hooks (eslint-plugin-react-hooks) — versão ^7.0.1

11.5.4 ESLint Plugin React Refresh (eslint-plugin-react-refresh) — versão ^0.4.26

11.5.5 Globals (globals) — versão ^16.5.0

11.6 Definições de Tipo (TypeScript Definitions)

11.6.1 @types/react — versão ^19.2.14

11.6.2 @types/react-dom — versão ^19.2.3

11.7 Backend-as-a-Service: Supabase

11.7.1 Visão Geral do Supabase

11.7.2 Cliente Supabase JS (@supabase/supabase-js) — versão ^2.95.3

11.7.3 PostgreSQL como Banco de Dados

11.7.4 Variáveis de Ambiente Relativas ao Supabase

11.8 Inteligência Artificial

11.8.1 Google Generative AI (@google/generative-ai) — versão ^0.24.1

11.8.2 Variável de Ambiente do Gemini

11.9 Manipulação de Dados e Exportação

11.9.1 xlsx (SheetJS) — versão ^0.18.5

11.10 Variáveis de Ambiente — Inventário Completo

11.11 Sistema de Versionamento e Colaboração

11.11.1 Git

11.11.2 GitHub

11.11.3 GitHub Actions / Workflows

11.12 Runtime e Empacotador

11.12.1 Node.js

11.12.2 Gerenciador de Pacotes

11.13 Estrutura de Diretórios do Projeto

11.14 Matriz Consolidada de Versões

11.15 Características Notáveis Ausentes (Considerações Arquiteturais)

11.16 Conclusão do Capítulo

RELATÓRIO DE INSPEÇÃO TÉCNICA — SEGUNDA EDIÇÃO

Relatório de Evolução da Plataforma de e-Learning Gamificada "Capacita Portos"

Documento: Relatório de Inspeção Técnica II (Relatório de Evolução) **Data de emissão:** 19 de maio de 2026 **Período analisado:** 6 de abril de 2026 a 19 de maio de 2026 **Documento de referência:** Relatório de Inspeção Técnica Exaustivo (1ª edição), de 6 de abril de 2026 **Versão do sistema na 1ª edição:** v0.0.0 **Versão do sistema nesta edição:** v0.0.0 (sem alteração de versionamento semântico) **Norma de referência:** ISO/IEC 25010:2015 — Modelo de Qualidade de Produto de Software **Classificação:** Confidencial — Uso Técnico Interno

SUMÁRIO EXECUTIVO PRELIMINAR

Este documento constitui a **segunda edição** do Relatório de Inspeção Técnica da plataforma de e-learning gamificada **Capacita Portos** (repositório `Treinamento`). Enquanto a primeira edição, datada de **6 de abril de 2026**, realizou uma fotografia completa da arquitetura, dos padrões de projeto, da segurança, da manutenibilidade e do desempenho do sistema, esta segunda edição tem natureza **incremental e comparativa**: seu objetivo é documentar, de forma exaustiva, **tudo o que foi adicionado, alterado ou corrigido na plataforma entre 6 de abril de 2026 e 19 de maio de 2026**.

No intervalo de aproximadamente seis semanas que separa as duas edições, a plataforma recebeu **cinco entregas de funcionalidade** (commits de feature), totalizando **2.077 linhas de código inseridas e 50 linhas removidas**, distribuídas por **16 arquivos** — sem contar o arquivo do primeiro relatório e o arquivo de lockfile de dependências. As entregas abrangem desde scripts de migração de dados em massa até um novo subsistema de controle de acesso por níveis, exportação de relatórios em formato Excel, moderação administrativa do fórum e melhorias na ferramenta de autoria de quizzes.

As cinco entregas analisadas em detalhe neste relatório são:

1. **Gestão de usuários em massa e enforcement de troca de senha** (scripts SQL e gerador Python);
2. **Sistema de Níveis de Acesso** (Básico / Intermediário / Avançado) com gating de conteúdo;
3. **Exportação de relatórios de alunos para Excel** (planilhas `.xlsx`);
4. **Moderação administrativa do fórum** (políticas de exclusão de posts e respostas);
5. **Gerenciamento dinâmico de opções de quiz** (adicionar e remover alternativas).

O presente relatório detalha cada uma dessas entregas em capítulo próprio, analisando motivação, implementação, impacto arquitetural, implicações de segurança, pontos de atenção e recomendações. Ao final, apresenta uma **análise de impacto consolidada** e uma **reavaliação dos índices de qualidade** estabelecidos na primeira edição.

1. INTRODUÇÃO E CONTEXTO

1.1 Objetivo do Documento

O objetivo deste documento é registrar, de maneira técnica, rastreável e auditável, a **evolução da plataforma Capacita Portos** desde a emissão do primeiro relatório de inspeção. Trata-se de um documento de **continuidade**: ele não substitui a primeira edição, mas a complementa, partindo do pressuposto de que o leitor tem acesso ao relatório anterior ou está familiarizado com o estado da plataforma em 6 de abril de 2026.

São objetivos específicos deste relatório:

- **Inventariar** todas as alterações de código realizadas no período;
- **Descrever tecnicamente** cada nova funcionalidade ou correção;
- **Avaliar o impacto** das mudanças sobre a arquitetura, a segurança, a manutenibilidade e o desempenho;
- **Verificar a aderência** das novas implementações aos padrões e princípios identificados na primeira edição (SOLID, Clean Code, OWASP);
- **Atualizar** os índices de qualidade do sistema à luz das novas entregas;
- **Recomendar** ações corretivas e preventivas para o ciclo de desenvolvimento seguinte.

1.2 Relação com o Primeiro Relatório

A primeira edição do Relatório de Inspeção Técnica foi um documento de **caráter fundacional**, com sete capítulos cobrindo: (1) arquitetura e padrões de projeto; (2) fluxo lógico e ciclo de vida; (3) complexidade ciclomática e manutenibilidade; (4) auditoria de segurança e robustez; (5) conformidade com Clean Code; (6) desempenho e recursos; e (7) conclusões e recomendações.

Aquele documento estabeleceu uma **linha de base (baseline)** de qualidade, sintetizada nos seguintes índices:

Dimensão	Índice na 1ª edição
Manutenibilidade	70 / 100
Testabilidade	40 / 100
Segurança	75 / 100
Desempenho	75 / 100
Conformidade com SOLID	75 / 100
Conformidade com Clean Code	65 / 100

A primeira edição também apontou seis **pontos críticos**, que servem de referência para a verificação de regressão nesta segunda edição:

1. Ausência de suíte de testes automatizados (cobertura de 0%);
2. Exposição da chave de API do Gemini (`VITE_GEMINI_API_KEY`) no bundle de frontend;
3. Complexidade ciclomática elevada em funções-chave;
4. Tratamento de erros inconsistente, sem logging estruturado;
5. Ausência de tipagem estática (TypeScript);
6. Ausência de sincronização em tempo real (uso de polling a 30 segundos).

Um dos critérios de avaliação desta segunda edição é justamente verificar se as cinco novas entregas **agravaram, mantiveram ou mitigaram** cada um desses pontos críticos.

1.3 Escopo e Metodologia

O escopo desta análise compreende **exclusivamente as alterações de código** introduzidas no repositório entre 6 de abril e 19 de maio de 2026. A metodologia empregada foi:

1. **Levantamento do histórico de versionamento (Git):** extração da lista completa de commits posteriores à data do primeiro relatório, com seus respectivos metadados (hash, autor, data, mensagem).
2. **Análise diferencial (diff):** inspeção linha a linha de cada commit, identificando arquivos criados, modificados e removidos.
3. **Análise de código estática:** leitura do código-fonte resultante para avaliar conformidade com padrões de projeto, princípios SOLID e práticas de Clean Code.
4. **Análise de segurança:** avaliação de cada alteração contra o catálogo OWASP Top 10 (2021), com atenção especial às mudanças que envolvem persistência de dados, controle de acesso e exposição de informações pessoais.
5. **Análise de impacto:** consolidação dos efeitos das mudanças sobre as métricas de qualidade da linha de base.

O **período coberto** delimita-se da seguinte forma: o commit mais antigo considerado é o de **6 de abril de 2026** (mesma data do primeiro relatório), incluído por não ter sido contemplado naquela edição; o commit mais recente é o de **19 de maio de 2026**, último registro disponível no momento da emissão deste documento.

1.4 Período Coberto e Linha do Tempo

A tabela a seguir apresenta a linha do tempo completa das entregas analisadas, ordenada cronologicamente:

#	Hash	Data	Descrição da entrega
1	0894ccf	06/04/2026	Scripts SQL para atualização de e-mails e enforcement de troca de senha
2	25effa4	13/04/2026	Níveis de acesso e migração de banco de dados para papéis de usuário
3	ba5232f	30/04/2026	Exportação de relatórios de alunos para Excel e atualização de dependências
4	15fe225	19/05/2026	Migração para permitir que o admin exclua posts e respostas do fórum
5	e081751	19/05/2026	Funcionalidade de gerenciar opções de quiz com botões de adicionar e remover

Observa-se uma **cadência de aproximadamente uma entrega a cada duas semanas**, com uma concentração de duas entregas no mesmo dia (19/05/2026). O intervalo entre a entrega de 30/04 e a de 19/05 (19 dias corridos) é o maior do período, sugerindo uma pausa no ritmo de desenvolvimento ou um período dedicado a atividades não versionadas (planejamento, testes manuais, operação).

1.5 Quadro-Resumo das Alterações

O quadro a seguir consolida o volume de alterações por entrega, considerando apenas arquivos de código e configuração relevantes (exclui-se o arquivo da primeira edição do relatório e o lockfile `package-lock.json`, cuja variação é gerada automaticamente):

Entrega	Arquivos	Linhas +	Linhas –	Natureza
0894ccf — Scripts SQL	4	~1.602	~37	Operacional / Dados
25effa4 — Níveis de Acesso	6 (código)	~249	~4	Funcionalidade / Arquitetura
ba5232f — Exportação Excel	3 (código)	~139	~28	Funcionalidade
15fe225 — Moderação do Fórum	1	24	0	Segurança / Persistência
e081751 — Opções de Quiz	2	71	0	Funcionalidade / UX
Total (código)	16	~2.077	~50	—

Leitura do quadro: a entrega de scripts SQL ([0894ccf](#)) responde, isoladamente, por cerca de **77% das linhas inseridas** no período. Trata-se, contudo, de código **operacional e descartável** (scripts de migração de dados executados uma única vez), e não de código de aplicação permanente. Excluída essa entrega, o volume de **código de aplicação efetivamente incorporado ao produto** é de aproximadamente **483 linhas inseridas**, número modesto, porém de **alto valor funcional** — concentrado em funcionalidades visíveis ao usuário final e ao administrador.

2. VISÃO GERAL DAS ENTREGAS DO PERÍODO

2.1 Classificação das Mudanças por Categoria

As cinco entregas do período podem ser classificadas segundo a taxonomia abaixo, que cruza a **natureza técnica** da mudança com o **público beneficiado**:

Entrega	Categoria primária	Categoria secundária	Público beneficiado
Scripts SQL (0894ccf)	Operação de dados	Segurança (enforcement de senha)	Administração / Operação
Níveis de Acesso (25effa4)	Nova funcionalidade	Arquitetura / Autorização	Aluno / Administrador
Exportação Excel (ba5232f)	Nova funcionalidade	Relatórios / BI	Administrador
Moderação do Fórum (15fe225)	Segurança / Autorização	Persistência (RLS)	Administrador
Opções de Quiz (e081751)	Melhoria de UX	Autoria de conteúdo	Administrador

Observação analítica: quatro das cinco entregas beneficiam diretamente o **perfil administrador**. Apenas o sistema de Níveis de Acesso entrega valor diretamente ao **aluno** (na forma de novas abas de conteúdo). Isso indica que o período foi predominantemente dedicado a **maturar as ferramentas de gestão da plataforma** — exportação de dados, moderação, autoria — em detrimento de novas funcionalidades de aprendizagem. Esse direcionamento é coerente com uma fase de **operacionalização** do produto: a presença de scripts de inserção em massa de alunos reais (capítulo 3) sugere que a plataforma passou a ser efetivamente utilizada em produção no período, o que naturalmente desloca o esforço de desenvolvimento para as necessidades de quem opera o sistema.

2.2 Mapa de Impacto sobre os Subsistemas

A primeira edição identificou cinco contextos lógicos (Bounded Contexts) na plataforma. A tabela a seguir mapeia quais contextos foram afetados por cada entrega:

Contexto (1ª edição)	Scripts SQL	Níveis Acesso	Excel	Fórum	Quiz
Autenticação & Autorização	●	●	—	●	—
Aprendizado & Progresso	—	●	—	—	●
Comunicação (Fórum + Dúvidas)	—	—	—	●	—
Inteligência Artificial	—	—	—	—	—
Administração	●	●	●	●	●

Legenda: ● contexto afetado pela entrega.

Leitura do mapa: o contexto de **Administração** foi tocado por **todas** as cinco entregas, confirmando o diagnóstico da seção anterior. O contexto de **Inteligência Artificial** (chat com Gemini) **não recebeu nenhuma alteração** no período — permanece, portanto, exatamente no estado descrito pela primeira edição, inclusive no que se refere à vulnerabilidade crítica de exposição da chave de API, ainda **não corrigida** (ver capítulo 8).

2.3 Arquivos Novos versus Arquivos Modificados

O período introduziu **8 arquivos inteiramente novos** e modificou **8 arquivos preexistentes**:

Arquivos novos:

Arquivo	Entrega	Tipo
<code>generate_sql.py</code>	0894ccf	Script Python (gerador)
<code>supabase/ sql_forcar_troca_senha_novos_alunos_2026_03_31.sql</code>	0894ccf	Script SQL
<code>supabase/sql_inserir_novos_alunos_2026_03_30.sql</code>	0894ccf	Script SQL
<code>src/lib/accessLevels.js</code>	25effa4	Módulo de domínio (JS)
<code>supabase/migration_access_levels.sql</code>	25effa4	Migração SQL
<code>supabase/migration_forum_admin_delete.sql</code>	15fe225	Migração SQL
(o arquivo do 1º relatório também foi adicionado em 25effa4)	25effa4	Documentação

Arquivos modificados:

Arquivo	Entrega(s)	Natureza da modificação
supabase/sql_trocar_emails_2026_03_30.sql	0894ccf	Expansão massiva (+602 linhas)
src/contexts/AuthContext.jsx	25effa4	Novo estado <code>accessLevel</code>
src/pages/DisciplineDetail.jsx	25effa4	Novas abas condicionais
src/pages/DisciplineDetail.css	25effa4	Estilos dos novos painéis
src/pages/admin/AdminUsers.jsx	25effa4	Coluna de nível de acesso
src/pages/admin/AdminReports.jsx	ba5232f	Função de exportação Excel
src/pages/admin/AdminReports.css	ba5232f	Estilos do botão de exportação
src/pages/admin/AdminDisciplineEdit.jsx	e081751	Gerenciamento de opções
src/pages/admin/AdminDisciplineEdit.css	e081751	Estilos dos botões de opção
package.json	ba5232f	Nova dependência <code>xlsx</code>

Observação: a relação equilibrada entre arquivos novos e modificados, somada ao fato de que nenhuma das modificações exigiu **reescrita estrutural** de arquivos preexistentes (todas foram aditivas), indica que a **arquitetura definida na primeira edição se mostrou suficientemente extensível** para absorver as cinco entregas sem refatoração. Esse é um sinal positivo de qualidade arquitetural — em conformidade com o **Princípio Aberto/Fechado (Open/Closed Principle)**, segundo o qual o software deve estar aberto para extensão e fechado para modificação.

3. ENTREGA 1 — GESTÃO DE USUÁRIOS EM MASSA E ENFORCEMENT DE TROCA DE SENHA

Commit: `0894ccf` — "Add SQL scripts for user email updates and password reset enforcement"

Data: 6 de abril de 2026 **Arquivos:** `generate_sql.py` (novo, 783 linhas), `supabase/sql_forcar_troca_senha_novos_alunos_2026_03_31.sql` (novo, 40 linhas), `supabase/sql_inserir_novos_alunos_2026_03_30.sql` (novo, 214 linhas), `supabase/sql_trocar_emails_2026_03_30.sql` (modificado, +602 linhas)

3.1 Contexto e Motivação

Esta entrega é de natureza distinta das demais: não introduz código de aplicação, mas sim **ferramental operacional de dados**. Seu propósito é viabilizar a **migração e o provisionamento em massa de contas de usuários reais** na plataforma. A presença de endereços de e-mail institucionais com domínio `@portosrio.gov.br`, ao lado de centenas de endereços pessoais (`@gmail.com`, `@hotmail.com`, `@icloud.com`), evidencia que a plataforma **transicionou de um ambiente de desenvolvimento para uso operacional real**, com uma turma concreta de alunos vinculados à autoridade portuária.

A motivação da entrega decorre de três necessidades operacionais simultâneas:

1. **Correção/padronização de e-mails:** alunos haviam sido cadastrados com endereços pessoais e precisavam ter seus e-mails atualizados — possivelmente para endereços institucionais ou para corrigir erros de digitação no cadastro original.
2. **Inserção de novos alunos:** uma nova leva de estudantes precisava ser provisionada no banco de autenticação do Supabase.
3. **Segurança de primeiro acesso:** os novos alunos, criados com senha provisória, precisavam ser **obrigados a definir uma senha própria** no primeiro login.

3.2 O Gerador `generate_sql.py`

O arquivo `generate_sql.py` (783 linhas) é um **script gerador de código** escrito em Python. Sua função é tomar como entrada duas listas extensas de endereços de e-mail — uma de e-mails "antigos" e outra de e-mails "novos" — e produzir, de forma automatizada, os comandos SQL de atualização.

A análise estática do arquivo revela um conteúdo dominado por **dados literais**: o script contém aproximadamente **767 ocorrências de endereços de e-mail** embutidas diretamente no código-fonte, organizadas em blocos de texto multilinha (`old_emails = ""..."`). Trata-se, portanto, de um script de **uso único e descartável**, cujo valor é puramente operacional e cuja vida útil se encerrou no momento de sua execução.

Análise técnica e pontos de atenção:

- **Dados pessoais em repositório de código (PII em VCS)**: a prática de embutir centenas de endereços de e-mail reais — informação pessoal identificável (PII) — diretamente no código-fonte versionado é o **ponto de atenção mais significativo desta entrega**. Uma vez incorporados ao histórico do Git, esses dados tornam-se **permanentes e imutáveis**: ainda que removidos em um commit futuro, permanecerão recuperáveis no histórico. Isso configura risco sob a ótica da **Lei Geral de Proteção de Dados (LGPD)**, especialmente considerando que parte dos titulares são servidores públicos identificáveis.
- **Acoplamento de dados e lógica**: o script mistura, no mesmo arquivo, a **lógica de geração** e os **dados a serem processados**. O ideal arquitetural seria que as listas de e-mails residissem em arquivos de dados externos (CSV, por exemplo) **não versionados** (incluídos no `.gitignore`), e que o script lesse esses arquivos como entrada.
- **Ausência de idempotência verificável**: não há, no gerador, mecanismo evidente que impeça a reexecução acidental do SQL gerado.

3.3 Script de Atualização de E-mails — `sql_trocar_emails_2026_03_30.sql`

Este arquivo recebeu a maior expansão da entrega: **+602 linhas**. Ele contém os comandos `UPDATE` sobre a tabela `auth.users` do Supabase, substituindo endereços de e-mail antigos por novos, registro a registro.

Pontos de atenção:

- A manipulação direta do schema `auth` do Supabase é uma operação **sensível e privilegiada**. Alterações na tabela `auth.users` afetam diretamente a capacidade de login dos usuários; um erro de mapeamento (associar o e-mail errado ao registro errado) pode resultar em **sequestro acidental de conta** — um aluno passando a ter acesso à conta de outro.
- A operação deveria, idealmente, estar envolvida em um bloco transacional (`BEGIN; ... COMMIT;`) com uma etapa de verificação (`SELECT` de conferência) antes do `COMMIT` , permitindo `ROLLBACK` em caso de divergência.

3.4 Script de Inserção de Novos Alunos —

`sql_inserir_novos_alunos_2026_03_30.sql`

Este arquivo (214 linhas) realiza a **inserção em massa de novos registros de alunos** no banco de autenticação. Cada novo aluno é criado já com a marcação de **obrigatoriedade de troca de senha** no primeiro acesso, conforme indicado pela mensagem do commit.

Análise técnica:

- A inserção direta em `auth.users` contorna o fluxo normal de cadastro da aplicação (`signUp` do AuthContext, descrito na primeira edição). Isso significa que **gatilhos, validações e efeitos colaterais** eventualmente associados ao cadastro pela interface **não são disparados** — é responsabilidade do script garantir que todos os campos obrigatórios e metadados sejam preenchidos corretamente.
- A criação de contas com **senha provisória conhecida** (necessária para o primeiro login) é um vetor de risco durante a janela entre a criação da conta e o primeiro acesso do aluno. O enforcement de troca de senha (seção seguinte) é justamente a mitigação desse risco.

3.5 Script de Enforcement de Troca de Senha —

`sql_forcar_troca_senha_novos_alunos_2026_03_31.sql`

Este é o script tecnicamente mais limpo da entrega (40 linhas) e o que mais se relaciona diretamente com a **segurança da plataforma**. Seu mecanismo é elegante e merece descrição detalhada:

```
BEGIN;

UPDATE auth.users u
SET
  raw_user_meta_data = COALESCE(u.raw_user_meta_data, '{}':jsonb)
                        || '{"must_reset_password": true}':jsonb,
  updated_at = NOW()
WHERE lower(u.email) IN ( ... lista de e-mails ... );

COMMIT;
```

Análise técnica — pontos positivos:

- **Uso de transação:** o script envolve a operação em `BEGIN; ... COMMIT;`, garantindo atomicidade — ou todos os registros são atualizados, ou nenhum é.
- **Mesclagem segura de JSONB:** o uso do operador de concatenação `||` sobre `jsonb`, combinado com `COALESCE(..., '{}':jsonb)`, garante que a flag `must_reset_password` seja **adicionada** ao metadado existente sem **sobrescrever** outros metadados eventualmente presentes (como `full_name`). É a abordagem correta.

- **Normalização de comparação:** o uso de `lower(u.email)` na cláusula `WHERE` torna a correspondência **case-insensitive**, evitando que diferenças de capitalização causem falha no enforcement.
- **Etapa de verificação:** o script inclui, após o `COMMIT`, uma consulta `SELECT` que lista os e-mails afetados e o valor resultante da flag `must_reset_password`, permitindo a **conferência manual** do resultado.

Integração com a aplicação: este mecanismo conecta-se diretamente à lógica de autenticação descrita na primeira edição. A flag `must_reset_password` em `raw_user_meta_data` é lida pela função `shouldResetPassword()` no `AuthContext`, que por sua vez alimenta o estado `mustResetPassword`. O componente `ProtectedRoute` consulta esse estado e **redireciona compulsoriamente** o usuário para a rota `/redefinir-senha` enquanto a flag estiver ativa. Trata-se, portanto, de uma entrega que **opera sobre uma funcionalidade já existente**, apenas ativando-a para um conjunto específico de novos usuários por via de dados, e não de código.

3.6 Análise de Segurança e Privacidade da Entrega

Aspecto	Avaliação	Comentário
Enforcement de senha	✓ Positivo	Mecanismo correto, transacional, case-insensitive
Atomicidade das operações	⚠ Parcial	Presente no script de senha; não confirmada nos demais
PII em repositório versionado	✗ Crítico	~767 e-mails reais embutidos no histórico do Git
Senhas provisórias	⚠ Atenção	Janela de risco mitigada pelo enforcement
Manipulação direta de <code>auth.users</code>	⚠ Atenção	Operação privilegiada; exige verificação rigorosa
Idempotência / reexecução	⚠ Atenção	Sem proteção evidente contra reexecução

3.7 Recomendações Específicas da Entrega 1

1. **Remover dados pessoais do versionamento:** mover as listas de e-mails para arquivos externos não versionados; adicionar `*.sql` de dados e `generate_sql.py` ao `.gitignore`, ou movê-los para um repositório operacional segregado do código de aplicação.
2. **Avaliar limpeza de histórico:** considerar, em conjunto com o encarregado de dados (DPO), a viabilidade e a necessidade de reescrever o histórico do Git (`git filter-repo`) para expurgar a PII, ponderando o impacto sobre clones existentes.

3. **Padronizar transações:** garantir que todos os scripts de mutação de dados sigam o padrão `BEGIN / verificação / COMMIT`.
 4. **Documentar a execução:** registrar, fora do código, a data, o responsável e o resultado da execução de cada script, para fins de auditoria.
-

4. ENTREGA 2 — SISTEMA DE NÍVEIS DE ACESSO

Commit: `25effa4` — *"feat: add access levels and database migration for user roles"* **Data:** 13 de abril de 2026 **Arquivos:** `src/lib/accessLevels.js` (novo), `supabase/migration_access_levels.sql` (novo), `src/contexts/AuthContext.jsx` (modificado), `src/pages/DisciplineDetail.jsx` (modificado), `src/pages/DisciplineDetail.css` (modificado), `src/pages/admin/AdminUsers.jsx` (modificado)

Esta é a entrega de **maior relevância arquitetural** do período. Ela introduz na plataforma um **segundo eixo de autorização**, ortogonal ao eixo de papéis (roles) já existente.

4.1 Motivação e Modelo Conceitual

Até esta entrega, a plataforma possuía um único eixo de autorização: o **papel** do usuário (`role`), que poderia assumir os valores `user`, `monitor` ou `admin`. Esse eixo responde à pergunta *"o que este usuário pode fazer?"* (consumir conteúdo, moderar dúvidas, administrar a plataforma).

A Entrega 2 acrescenta um **eixo independente e ortogonal**: o **nível de acesso** (`access_level`), que pode assumir os valores `basico`, `intermediario` ou `avancado`. Esse novo eixo responde a uma pergunta diferente: *"quanto de conteúdo este usuário pode ver?"*. A ortogonalidade é importante: um mesmo aluno tem **um papel** e **um nível de acesso** simultaneamente, e os dois são gerenciados de forma independente.

O modelo de níveis é **estritamente hierárquico e cumulativo**, conforme documentado no cabeçalho da própria migração SQL:

Nível	Conteúdo liberado
Básico	Vídeos, apostila, quiz, fórum
Intermediário	Tudo do Básico + Atividade de Reflexão
Avançado	Tudo do Intermediário + Artigo e Texto Técnico

A natureza **cumulativa** significa que cada nível superior **engloba integralmente** os direitos do nível inferior — não há conteúdo exclusivo de um nível intermediário que seja vedado ao nível avançado. Esse desenho simplifica a lógica de verificação, que se reduz a uma comparação de ordem (*"o nível do usuário é maior ou igual ao nível exigido?"*).

4.2 Camada de Domínio — `src/lib/accessLevels.js`

O novo módulo `accessLevels.js` (18 linhas) é o coração lógico da funcionalidade. É um módulo **puro**, sem efeitos colaterais e sem dependências externas — uma escolha de design exemplar do ponto de vista da testabilidade. Seu conteúdo integral:

```
export const ACCESS_LEVELS = ['basico', 'intermediario', 'avancado']

export const ACCESS_LEVEL_LABELS = {
  basico: 'Básico',
  intermediario: 'Intermediário',
  avancado: 'Avançado',
}

const RANK = { basico: 0, intermediario: 1, avancado: 2 }

export function hasAccessLevel(userLevel, requiredLevel) {
  const u = RANK[userLevel] ?? 0
  const r = RANK[requiredLevel] ?? 0
  return u >= r
}

export const canSeeReflexao = (level) => hasAccessLevel(level, 'intermediario')
export const canSeeArtigoTecnico = (level) => hasAccessLevel(level, 'avancado')
```

Análise técnica detalhada:

- **Padrão de mapeamento de ordem (`RANK`)**: a constante `RANK` traduz os níveis textuais em valores numéricos ordenáveis. Essa é a técnica que viabiliza a comparação hierárquica `u >= r`. É uma implementação simples e correta do conceito de **ordem total** sobre um conjunto enumerado.
- **Robustez via *nullish coalescing* (`??`)**: a expressão `RANK[userLevel] ?? 0` é uma decisão de design defensiva importante. Caso o valor recebido seja `undefined`, `null` ou qualquer string desconhecida (por exemplo, em decorrência de dado corrompido ou de uma versão futura do enum), a função degrada com segurança para o **nível mais restritivo** (`0`, equivalente a `basico`). Esse comportamento de *fail-safe* (falhar para o estado seguro) é a escolha correta em código de autorização: na dúvida, **negar acesso**.
- **Funções semânticas de conveniência (`canSeeReflexao`, `canSeeArtigoTecnico`)**: ao invés de espalhar a string mágica `'intermediario'` pela base de código, o módulo expõe funções de **intenção clara**. O componente consumidor pergunta `canSeeReflexao(nivel)` em vez de `hasAccessLevel(nivel, 'intermediario')`. Isso melhora a legibilidade e **centraliza a regra de negócio**: se amanhã a Atividade de Reflexão passar a exigir nível avançado, a alteração ocorre em **um único ponto**.

- **Aderência ao princípio DRY e ao SRP:** o módulo tem uma **responsabilidade única** (definir e comparar níveis de acesso) e elimina duplicação de regra. Está em plena conformidade com os princípios destacados como desejáveis na primeira edição.

Ponto de atenção menor: a constante `ACCESS_LEVEL_LABELS` é exportada, mas o componente `AdminUsers.jsx` (seção 4.6) **redefine localmente** uma estrutura equivalente (`ACCESS_LEVELS` com `value / label`). Há, portanto, uma **pequena duplicação** entre o módulo de domínio e o componente de administração. O ideal seria que `AdminUsers.jsx` consumisse `ACCESS_LEVELS` e `ACCESS_LEVEL_LABELS` do módulo, evitando que as duas listas divergam no futuro.

4.3 Camada de Persistência — `supabase/migration_access_levels.sql`

A migração `migration_access_levels.sql` (101 linhas) implementa o suporte do novo eixo de autorização no banco de dados PostgreSQL do Supabase. A migração é **bem documentada** (cabeçalho explicativo com as regras de negócio) e estruturada em **seis etapas numeradas**:

4.3.1 Etapa 1 — Criação do Tipo Enumerado

```
DO $$
BEGIN
  IF NOT EXISTS (SELECT 1 FROM pg_type WHERE typename = 'access_level_enum') THEN
    CREATE TYPE access_level_enum AS ENUM ('basico', 'intermediario', 'avancado');
  END IF;
END$$;
```

Cria um tipo `ENUM` nativo do PostgreSQL. O envoltório `DO $$ ... IF NOT EXISTS ... $$` torna a criação **idempotente** — a migração pode ser reexecutada sem erro. O uso de `ENUM` no nível do banco é uma decisão sólida: garante **integridade referencial** (é impossível inserir um valor fora dos três permitidos) diretamente na camada de persistência.

4.3.2 Etapa 2 — Adição da Coluna

```
ALTER TABLE user_roles
  ADD COLUMN IF NOT EXISTS access_level access_level_enum NOT NULL DEFAULT 'basico';
```

Acrescenta a coluna `access_level` à tabela `user_roles`. As cláusulas `IF NOT EXISTS` (idempotência), `NOT NULL` (integridade) e `DEFAULT 'basico'` (valor seguro padrão) estão todas presentes — implementação correta. O default `basico` garante que todo usuário novo nasça no **nível mais restritivo**, em coerência com o princípio de menor privilégio.

4.3.3 Etapa 3 — Retrocompatibilidade de Dados

```
INSERT INTO user_roles (user_id, role, access_level)
SELECT u.id, 'user', 'basico'
FROM auth.users u
LEFT JOIN user_roles r ON r.user_id = u.id
WHERE r.user_id IS NULL
ON CONFLICT (user_id) DO NOTHING;
```

Esta etapa é especialmente cuidadosa: ela garante que **todos os usuários já existentes** que ainda não possuíam linha em `user_roles` recebam uma, com `role = 'user'` e `access_level = 'basico'`. O `LEFT JOIN ... WHERE r.user_id IS NULL` é o padrão idiomático para "encontrar registros órfãos", e o `ON CONFLICT DO NOTHING` previne erro em caso de corrida ou reexecução. É uma **migração de dados defensiva e correta**.

4.3.4 Etapa 4 — Função de Leitura do Próprio Nível

```
CREATE OR REPLACE FUNCTION get_my_access_level()
RETURNS TEXT AS $$
DECLARE lvl TEXT;
BEGIN
    SELECT access_level::TEXT INTO lvl FROM user_roles WHERE user_id = auth.uid();
    RETURN COALESCE(lvl, 'basico');
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;
```

Função RPC que permite a um usuário consultar **o próprio** nível de acesso. Usa `auth.uid()` para identificar o chamador — ou seja, é impossível um usuário consultar o nível de outro por meio dela. O `COALESCE(lvl, 'basico')` reitera o padrão *fail-safe*.

Observação relevante: apesar de corretamente implementada, esta função **não é utilizada pelo frontend** (ver seção 4.4). O `AuthContext` obtém o nível de acesso por outra via. Trata-se, portanto, de **código morto** no momento — uma função pronta para uso, porém ainda não consumida. Não é um defeito, mas merece registro: ou o frontend deveria migrar para usá-la, ou ela deveria ser removida para evitar confusão futura.

4.3.5 Etapa 5 — Função de Escrita (Administrativa)

```
CREATE OR REPLACE FUNCTION set_user_access_level(p_user_id UUID, p_access_level TEXT)
RETURNS VOID AS $$
BEGIN
    IF NOT is_admin() THEN
        RAISE EXCEPTION 'Acesso negado: apenas administradores podem alterar níveis de acesso';
    END IF;
    IF p_access_level NOT IN ('basico', 'intermediario', 'avancado') THEN
        RAISE EXCEPTION 'Nível de acesso inválido: %', p_access_level;
    END IF;
    INSERT INTO user_roles (user_id, role, access_level)
    VALUES (p_user_id, 'user', p_access_level::access_level_enum)
    ON CONFLICT (user_id) DO UPDATE SET access_level = EXCLUDED.access_level;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;
```

Esta é a função **mais sensível** da migração, pois realiza uma operação de **escalonamento de privilégio**. Sua análise de segurança é detalhada na seção 4.8. Em resumo, ela implementa **duas barreiras de validação** antes de qualquer escrita: (1) verificação de que o chamador é administrador (`is_admin()`); (2) verificação de que o valor recebido é um dos três níveis válidos. Ambas as falhas resultam em `RAISE EXCEPTION`, abortando a operação. O `ON CONFLICT ... DO UPDATE` torna a função um **upsert**, funcionando tanto para usuários com linha preexistente quanto para os sem.

4.3.6 Etapa 6 — Atualização da Função `get_platform_users`

A migração faz `DROP` e recria a função `get_platform_users()` para que ela passe a retornar duas novas colunas: `role` e `access_level`. Essa função é a fonte de dados da tela administrativa de usuários. Ela também é protegida por `IF NOT is_admin() THEN RAISE EXCEPTION`, e usa `LEFT JOIN` com `COALESCE` para garantir que usuários sem linha em `user_roles` apareçam com valores-padrão (`user` / `basico`) em vez de `NULL`.

4.4 Camada de Estado — `src/contexts/AuthContext.jsx`

O `AuthContext` foi modificado para **propagar o nível de acesso** a toda a aplicação, exatamente como já fazia com `isAdmin`, `isMonitor` e `userRole`. As alterações (10 linhas) foram:

1. **Novo estado:** `const [accessLevel, setAccessLevel] = useState('basico')` — inicializado, corretamente, no nível mais restritivo.
2. **Leitura no `checkRoles`:** a consulta a `user_roles` foi expandida de `.select('role')` para `.select('role, access_level')`, e o resultado alimenta `setAccessLevel(data?.access_level || 'basico')`.

3. **Caso administrador:** quando o usuário é o administrador (identificado por e-mail), o nível é fixado em `setAccessLevel('avancado')` — coerente, pois o administrador deve enxergar todo o conteúdo.
4. **Casos de limpeza:** em todos os pontos de *reset* do contexto (usuário nulo, falha na consulta, logout), `accessLevel` é restaurado para `'basico'`.
5. **Exposição no Provider:** `accessLevel` foi acrescentado ao objeto `value` do `AuthContext.Provider`, tornando-se consumível por qualquer componente via `useAuth()`.

Análise técnica:

- A alteração é **minimalista, consistente e completa** — todos os pontos do ciclo de vida do contexto foram cobertos, sem deixar caminho em que `accessLevel` pudesse ficar dessincronizado. Está em conformidade com o **padrão Observer** descrito na primeira edição.
- **Decisão de design notável:** o frontend lê `access_level` **diretamente da tabela** `user_roles` (via `select`), e **não** por meio da RPC `get_my_access_level()` criada na migração. Funcionalmente o resultado é o mesmo, mas há uma **inconsistência de abordagem**: a migração investiu em uma função RPC dedicada que acabou não sendo o caminho adotado. Recomenda-se uniformizar — ou o frontend passa a usar a RPC, ou a RPC é removida.
- **Dependência de RLS:** para que o `select` direto sobre `user_roles` funcione, é imperativo que exista uma política de Row Level Security permitindo ao usuário **ler a própria linha** de `user_roles`. Caso essa política não exista (a migração analisada não a cria), a consulta retornará vazio e **todos os usuários cairão silenciosamente no nível básico** por força do `|| 'basico'`. Recomenda-se a verificação explícita da existência dessa política RLS.

4.5 Camada de Apresentação — `src/pages/DisiplineDetail.jsx`

A página de detalhe da disciplina foi a que mais visivelmente mudou para o aluno. As alterações (53 linhas) introduzem **duas novas abas condicionais** na navegação da disciplina:

1. **Aba "Atividade de Reflexão"** (ícone `FiEdit3`) — renderizada apenas se `canSeeReflexao(accessLevel)` for verdadeiro (nível intermediário ou superior).
2. **Aba "Artigo e Texto Técnico"** (ícone `FiBookOpen`) — renderizada apenas se `canSeeArtigoTecnico(accessLevel)` for verdadeiro (nível avançado).

O fluxo de implementação:

```
const { user, isAdmin, isMonitor, accessLevel } = useAuth()
const showReflexao = canSeeReflexao(accessLevel)
const showArtigoTecnico = canSeeArtigoTecnico(accessLevel)
```

As duas variáveis booleanas `showReflexao` e `showArtigoTecnico` são computadas uma única vez no topo do componente e usadas tanto para renderizar o **botão da aba** quanto para renderizar o **painel de conteúdo** correspondente. A renderização condicional usa o padrão idiomático do React `{showReflexao && ( ... )}`.

Análise técnica:

- **Dupla guarda de renderização:** tanto a aba quanto o painel verificam a condição (`{showReflexao && ...}` na aba e `{activeTab === 'reflexao' && showReflexao && ...}` no painel). A repetição da verificação no painel é uma **redundância defensiva saudável**: impede que o painel seja exibido caso `activeTab` seja manipulado por outro caminho.
- **Conteúdo ainda não implementado:** ambos os painéis exibem, no momento, apenas um **estado vazio** ("Em breve: conteúdo da atividade de reflexão desta disciplina."). Ou seja, esta entrega implementa a **estrutura de gating e de navegação**, mas o **conteúdo real ainda não existe**. A funcionalidade está, portanto, em estado de **andaime (scaffold)** — pronta para receber o conteúdo, mas ainda sem entregá-lo ao aluno. Isso deve ser comunicado claramente às partes interessadas para não gerar expectativa equivocada.
- **Gating é puramente client-side — alerta importante:** a decisão de exibir ou ocultar as abas ocorre **inteiramente no navegador**. No estado atual isso é **inofensivo**, pois não há conteúdo sensível por trás das abas (apenas mensagens "Em breve"). Contudo, **quando o conteúdo real for adicionado**, o controle de acesso **não poderá depender apenas** desta verificação de frontend: um usuário de nível básico poderia, em tese, inspecionar o código ou manipular o estado para revelar as abas. O conteúdo real **deverá ser protegido também no nível do banco de dados** (via RLS, condicionada ao `access_level`), de modo que a consulta de dados retorne vazio para quem não tem nível suficiente. Este é o **principal débito técnico embutido nesta entrega** e precisa ser endereçado **antes** da publicação de conteúdo real nas novas abas.
- **Acréscimo de estados ao `DisciplineDetail`:** a primeira edição já havia sinalizado que `DisciplineDetail` é um componente sobrecarregado, com "20+ estados". Esta entrega acrescenta nova lógica de aba (`activeTab` passa a ter dois valores possíveis a mais). O incremento é pequeno, mas reforça a recomendação anterior de **refatorar `DisciplineDetail`** — eventualmente extraindo a lógica de abas para um componente ou hook dedicado.

O arquivo `DisciplineDetail.css` recebeu 38 linhas de estilo para os novos painéis (`.reflexao-panel`, `.artigo-panel`), com cabeçalho, corpo e a cor institucional `#009b8f` já usada no restante da aplicação — **consistência visual preservada**.

4.6 Camada de Administração — `src/pages/admin/AdminUsers.jsx`

A tela administrativa de usuários ganhou uma **nova coluna "Nível de Acesso"** na tabela de usuários, contendo um `<select>` que permite ao administrador alterar o nível de cada aluno individualmente. As alterações (33 linhas) incluem:

- A constante local `ACCESS_LEVELS` com os pares `value / label` ;
- A função `handleAccessLevelChange(userId, newLevel)` ;
- A nova coluna `<th>Nível de Acesso</th>` e a célula `<td>` com o `<select>` .

A função `handleAccessLevelChange` merece destaque por implementar o padrão de **atualização otimista (optimistic update)**:

```
const handleAccessLevelChange = async (userId, newLevel) => {
  const previous = users
  setUsers((prev) =>
    prev.map((u) => (u.id === userId ? { ...u, access_level: newLevel } : u))
  )
  const { error: rpcError } = await supabase.rpc('set_user_access_level', {
    p_user_id: userId,
    p_access_level: newLevel,
  })
  if (rpcError) {
    setUsers(previous)
    setError('Não foi possível atualizar o nível de acesso: ' + rpcError.message)
  }
}
```

Análise técnica:

- **Atualização otimista bem implementada:** a função (1) guarda o estado anterior (`previous`); (2) atualiza a UI **imediatamente**, antes da resposta do servidor — proporcionando resposta instantânea ao administrador; (3) chama a RPC `set_user_access_level`; (4) em caso de erro, executa o **rollback** do estado para `previous` e exibe mensagem. Este é um padrão de UX **maduro e correto**, e representa uma **evolução qualitativa** em relação ao tratamento de erros descrito na primeira edição como "inconsistente": aqui o erro é **explicitamente capturado, revertido e comunicado ao usuário**.
- **Defesa em profundidade:** a UI delega a operação à RPC `set_user_access_level`, que — como visto na seção 4.3.5 — revalida no servidor tanto a permissão de administrador quanto a validade do valor. Ainda que um atacante forjasse a chamada, a barreira do banco permaneceria. Excelente exemplo de **defesa em camadas**.
- **Ponto de atenção — CSS ausente:** o `<select>` recebe a classe `access-level-select`, porém **nenhuma regra CSS com esse seletor foi adicionada** nesta entrega (verificado por busca na base de código). O elemento será renderizado com o estilo padrão do navegador.

É um **defeito cosmético menor**, sem impacto funcional, mas que deveria ser corrigido para manter a consistência visual da tela administrativa.

4.7 Análise de Padrões de Projeto da Entrega 2

A Entrega 2 reforça e estende vários dos padrões catalogados na primeira edição:

Padrão	Como a Entrega 2 o utiliza
Observer	<code>accessLevel</code> integra-se ao <code>AuthContext</code> , propagando-se a todos os assinantes via <code>useAuth()</code>
Strategy	<code>canSeeReflexao</code> e <code>canSeeArtigoTecnico</code> encapsulam estratégias de verificação de visibilidade
Guard Clause	A migração SQL usa <code>IF NOT is_admin() THEN RAISE EXCEPTION</code> como cláusula de guarda
Fail-Safe Default	<code>?? 0</code> e <code> 'basico'</code> garantem degradação para o estado mais restritivo
Optimistic UI	<code>handleAccessLevelChange</code> aplica e, se preciso, reverte a mudança

A entrega adiciona, portanto, **valor arquitetural líquido positivo**: introduz uma capacidade nova (autorização em segundo eixo) reaproveitando a infraestrutura existente, sem violar nenhum dos padrões estabelecidos.

4.8 Análise de Segurança da Entrega 2

Vetor	Avaliação	Comentário
Escalonamento de privilégio via RPC	✓ Mitigado	<code>set_user_access_level</code> exige <code>is_admin()</code> no servidor
Injeção de valor inválido	✓ Mitigado	Validação <code>NOT IN (...)</code> + tipo <code>ENUM</code> no banco
Exposição de dados de outros usuários	✓ Mitigado	<code>get_my_access_level</code> usa <code>auth.uid()</code> ; <code>get_platform_users</code> exige admin
Gating de conteúdo no cliente	⚠ Débito técnico	Aceitável hoje (sem conteúdo real); exigirá RLS quando houver conteúdo
Política RLS de leitura de <code>user_roles</code>	⚠ A verificar	Frontend depende de RLS de auto-leitura não confirmada na migração
<code>SECURITY DEFINER</code>	✓ Adequado	Necessário e corretamente acompanhado de verificação de papel

O uso de `SECURITY DEFINER` nas funções da migração é apropriado: essas funções precisam de privilégios elevados para ler `auth.users` e escrever em `user_roles`, e **todas** as funções que executam operações sensíveis verificam `is_admin()` **antes** de qualquer ação. Esta é a forma correta de usar `SECURITY DEFINER` — o risco desse modificador é justamente expor privilégios sem checagem, o que **não** ocorre aqui.

4.9 Recomendações Específicas da Entrega 2

1. **(Prioridade alta) Planejar a proteção server-side do conteúdo das novas abas** antes de publicar qualquer material real de Reflexão ou Artigo Técnico — via políticas RLS condicionadas a `access_level`.
 2. **Verificar a política RLS de auto-leitura de `user_roles`**, da qual depende o carregamento do nível no `AuthContext`.
 3. **Eliminar a duplicação de listas de níveis:** fazer `AdminUsers.jsx` consumir `ACCESS_LEVELS / ACCESS_LEVEL_LABELS` de `accessLevels.js`.
 4. **Decidir o destino da RPC `get_my_access_level()`**: adotá-la no frontend ou removê-la.
 5. **Adicionar o estilo CSS `.access-level-select`** ausente em `AdminUsers.jsx`.
 6. **Aproveitar a pureza de `accessLevels.js`** para escrever os **primeiros testes unitários** da plataforma (ver capítulo 8) — a função `hasAccessLevel` é trivialmente testável e seria um excelente ponto de partida para reverter a cobertura de 0%.
-

5. ENTREGA 3 — EXPORTAÇÃO DE RELATÓRIOS DE ALUNOS PARA EXCEL

Commit: `ba5232f` — *"feat: add Excel export functionality for student reports and update dependencies"* **Data:** 30 de abril de 2026 **Arquivos:** `src/pages/admin/AdminReports.jsx` (modificado), `src/pages/admin/AdminReports.css` (modificado), `package.json` (modificado), `package-lock.json` (modificado)

5.1 Motivação

A primeira edição descreveu a página `AdminReports` como o painel administrativo de estatísticas, contendo a função `getDisciplineMetrics()` — apontada, à época, como uma das funções de **maior complexidade ciclomática** do sistema (CC = 8). Aquele painel já consolidava métricas de progresso dos alunos **dentro da própria interface web**.

A motivação da Entrega 3 é uma necessidade recorrente em qualquer plataforma educacional operada institucionalmente: **levar os dados para fora da aplicação**. Gestores e coordenadores frequentemente precisam manipular dados de desempenho em planilhas — para cruzar com outras informações, gerar gráficos próprios, arquivar para auditoria ou apresentar a terceiros. A exportação para Excel atende a essa demanda transformando o painel, antes apenas **consultável**, em uma fonte de dados **exportável**.

5.2 Nova Dependência — `xlsx`

A entrega adiciona ao `package.json` a dependência de produção `xlsx` na versão `^0.18.5`:

```
"react-router-dom": "^7.13.0",  
"xlsx": "^0.18.5"
```

A biblioteca `xlsx` (também conhecida como SheetJS, na sua edição comunitária) é a solução mais difundida do ecossistema JavaScript para leitura e escrita de planilhas. A escolha é **pragmática e padrão de mercado**.

Pontos de atenção sobre a dependência:

- **Atualização do `package-lock.json`**: o commit altera 142 linhas do lockfile — variação esperada e gerada automaticamente pelo gerenciador de pacotes ao resolver a árvore de dependências transitivas de `xlsx`.
- **Versão e canal de distribuição**: a versão `0.18.5` é uma versão **anterior** à reestruturação de distribuição da biblioteca. Recomenda-se que a equipe **monitore os avisos de segurança** referentes a essa biblioteca e ao seu canal de distribuição, e avalie a migração para a versão mais recente publicada pelo mantenedor. Em particular, versões antigas de `xlsx` foram alvo de avisos relativos a *prototype pollution* e a *Regular Expression Denial of Service* (ReDoS) no caminho de **leitura** de arquivos. Como esta entrega usa a biblioteca **exclusivamente para escrita** de arquivos (a aplicação **não** lê planilhas enviadas pelo usuário), a **superfície de exposição é reduzida** — mas o monitoramento permanece recomendável.
- **Impacto no tamanho do bundle**: a biblioteca `xlsx` é relativamente volumosa. Sua importação via `import * as XLSX from 'xlsx'` traz o módulo inteiro. Como `AdminReports` é uma página acessível apenas a administradores, recomenda-se avaliar o **carregamento sob demanda** (*lazy loading / code splitting*) dessa rota, para que o peso da biblioteca **não onere o carregamento inicial** percebido pelo aluno comum, que nunca acessa essa tela.

5.3 A Função `exportStudentsToExcel`

O núcleo da entrega é a nova função `exportStudentsToExcel` (cerca de 75 linhas) em `AdminReports.jsx`. Sua lógica desdobra-se em quatro fases:

5.3.1 Fase 1 — Seleção do Conjunto de Dados

```
const dataset = filteredUsers.length > 0 ? filteredUsers : users
```

A função respeita o **filtro de busca ativo** na tela: se o administrador digitou um termo de busca (filtrando a lista de alunos), a exportação contempla **apenas os alunos filtrados**; caso contrário, exporta **todos**. Essa decisão de design é acertada — o que se vê na tela é o que se exporta, princípio de menor surpresa (*principle of least astonishment*).

5.3.2 Fase 2 — Construção da Planilha "Resumo"

A função mapeia cada usuário do `dataset` para uma linha de resumo, reutilizando a função preexistente `getUserMetrics(u.id)`. Cada linha contém 11 colunas, com **cabeçalhos em português** e legíveis por humanos:

Coluna	Origem
Nome	<code>u.full_name</code>
Email	<code>u.email</code>
Cadastrado em	<code>u.created_at</code> , formatado em <code>pt-BR</code>
Último acesso	<code>u.last_sign_in_at</code> , formatado em <code>pt-BR</code>
Disciplinas concluídas	<code>getUserMetrics</code>
Disciplinas em andamento	<code>getUserMetrics</code>
Total de disciplinas na plataforma	<code>disciplines.length</code>
Aulas concluídas	<code>getUserMetrics</code>
Quizzes finais realizados	<code>getUserMetrics</code>
Quizzes de aula realizados	<code>getUserMetrics</code>
Média do quiz final (%)	<code>getUserMetrics</code>

5.3.3 Fase 3 — Construção da Planilha "Detalhado"

A segunda planilha desce ao **grão da disciplina**: para cada par (aluno × disciplina), gera uma linha — **desde que haja atividade** naquele par. O filtro de atividade é:

```
const hasActivity = d.lessonsCompleted > 0 || d.quizScore !== null || d.lessonQuizzes.length
if (!hasActivity) return
```

Esse filtro é uma **otimização inteligente**: evita poluir a planilha com milhares de linhas de pares aluno×disciplina sem nenhuma interação. A planilha detalhada contém 13 colunas, incluindo status textual da disciplina ("Concluída", "Em andamento", "Não iniciada"), progresso percentual, contagem de quizzes de aula aprovados e realizados, e dados granulares do quiz final (percentual, acertos, total de questões, data de conclusão).

5.3.4 Fase 4 — Montagem e Escrita do Arquivo

```
const wb = XLSX.utils.book_new()
const wsResumo = XLSX.utils.json_to_sheet(summaryRows)
const wsDetalhado = XLSX.utils.json_to_sheet(detailRows)
// ... definição de larguras de coluna via ws['!cols'] ...
XLSX.utils.book_append_sheet(wb, wsResumo, 'Resumo')
XLSX.utils.book_append_sheet(wb, wsDetalhado, 'Detalhado')
const today = new Date().toISOString().slice(0, 10)
XLSX.writeFile(wb, `relatorio-alunos-${today}.xlsx`)
```

A função cria uma pasta de trabalho (*workbook*) com **duas abas** ("Resumo" e "Detalhado"), define **larguras de coluna explícitas** (via `ws['!cols']`) para que a planilha já abra legível, e dispara o download com um **nome de arquivo datado** (`relatorio-alunos-AAAA-MM-DD.xlsx`).

5.4 Avaliação da Qualidade da Implementação

Pontos fortes:

- **Reuso de lógica existente:** a função **não reimplementa** o cálculo de métricas — ela reaproveita `getUserMetrics` e `getUserDisciplineDetail`, já presentes no componente. Isso garante que **os números do Excel coincidam com os números da tela**, eliminando o risco de divergência entre dois caminhos de cálculo. É uma decisão de design corretíssima e em plena aderência ao princípio **DRY**.
- **Cuidado com a apresentação:** cabeçalhos em português, datas formatadas no padrão brasileiro, larguras de coluna pré-ajustadas e duas abas com granularidades distintas — a entrega demonstra **atenção à experiência do usuário final** do relatório, e não apenas ao despejo bruto de dados.
- **Nome de arquivo datado:** facilita o arquivamento e evita sobrescrita acidental de exportações anteriores.
- **Tratamento de valores ausentes:** o uso sistemático de `?? ''` e `|| ''` evita que células fiquem com `undefined` ou `null` literais.

Pontos de atenção:

- **Operação síncrona e bloqueante:** a montagem das linhas e a escrita do arquivo ocorrem de forma **síncrona, na thread principal**. Para o volume atual de alunos isso é imperceptível. Porém, considerando que os scripts da Entrega 1 indicam **centenas de alunos** cadastrados, e que a planilha detalhada gera uma linha por par aluno×disciplina, o número de linhas pode crescer para a casa dos milhares. Em volumes elevados, a geração síncrona pode **congelar a interface por alguns segundos**. Recomenda-se, como evolução futura, exibir um **indicador de carregamento** durante a geração e, se necessário, mover o processamento para um *Web Worker*.
- **Ausência de tratamento de erro:** diferentemente de `handleAccessLevelChange` (Entrega 2), a função `exportStudentsToExcel` **não possui bloco `try/catch`**. Caso `XLSX.writeFile` falhe (por exemplo, em um navegador com restrição de download), a exceção propagará sem tratamento e **sem feedback ao usuário**. Isso reincide no ponto crítico nº 4 da primeira edição ("tratamento de erros inconsistente"). Recomenda-se envolver a função em `try/catch` com mensagem de erro amigável.
- **Complexidade acrescida a `AdminReports.jsx`:** a primeira edição já classificava `getDisciplineMetrics()` como função crítica (CC = 8) neste mesmo arquivo. A adição de `exportStudentsToExcel` — que contém dois laços aninhados (`dataset.forEach` × `disciplines.forEach`) e várias condicionais — **aumenta a carga de complexidade do**

componente. O arquivo `AdminReports.jsx` consolida-se como um **candidato prioritário a refatoração**, idealmente extraindo a lógica de exportação para um módulo utilitário próprio (por exemplo, `src/lib/exportReports.js`), o que também o tornaria testável isoladamente.

5.5 Interface e Estilos

O arquivo `AdminReports.css` recebeu 35 linhas para estilizar o novo **botão "Exportar Excel"** (`.report-export-btn`) e o contêiner de ações (`.report-section-actions`). O botão:

- usa a cor institucional `#009b8f`, com estado `:hover` mais escuro (`#007a72`) e leve deslocamento no `:active` — micro-interações que conferem polimento;
- possui estado `:disabled` (cor esmaecida `#9ec9c5`, cursor `not-allowed`), acionado quando `users.length === 0` — **prevenção correta** de exportação de relatório vazio;
- inclui ícone `FiDownload` e o atributo `title` ("Exportar relatório de alunos para Excel"), contribuindo para a acessibilidade.

O reposicionamento do campo de busca para dentro de um novo contêiner `.report-section-actions` (com `display: flex` e `flex-wrap: wrap`) garante que busca e botão de exportação coabitem o cabeçalho da seção de forma **responsiva**.

5.6 Recomendações Específicas da Entrega 3

1. **Adicionar tratamento de erro** (`try/catch`) à função `exportStudentsToExcel`, com feedback visual ao administrador.
 2. **Extrair a lógica de exportação** para um módulo dedicado (`src/lib/exportReports.js`), reduzindo a complexidade de `AdminReports.jsx` e habilitando testes unitários.
 3. **Avaliar o carregamento sob demanda** da rota `AdminReports` (e, com ela, da biblioteca `xlsx`) para não onerar o bundle inicial.
 4. **Exibir indicador de progresso** durante a geração, antecipando o crescimento do volume de dados.
 5. **Monitorar avisos de segurança** da dependência `xlsx` e avaliar a atualização para a versão mais recente do mantenedor.
-

6. ENTREGA 4 — MODERAÇÃO ADMINISTRATIVA DO FÓRUM

Commit: `15fe225` — *"feat: add migration to allow admin to delete forum posts and replies"* **Data:** 19 de maio de 2026 **Arquivos:** `supabase/migration_forum_admin_delete.sql` (novo, 24 linhas)

6.1 Contexto e Motivação

A primeira edição descreveu o fórum como parte do **Contexto de Comunicação**, com as tabelas `forum_posts` e `forum_replies`. Em uma plataforma com centenas de alunos reais, o fórum é um espaço de discussão aberto e, como tal, **sujeito a conteúdo inadequado** — mensagens ofensivas, spam, conteúdo fora de tema ou publicado por engano. A capacidade de **moderação** torna-se, portanto, um requisito operacional.

Esta entrega é tecnicamente **pequena** (um único arquivo, 24 linhas), porém **conceitualmente importante**, e sua análise revela um detalhe arquitetural significativo.

6.2 As Políticas de Row Level Security

A migração cria duas políticas de Row Level Security (RLS) no PostgreSQL:

```
-- Forum Posts: admin pode deletar qualquer post
DROP POLICY IF EXISTS "Admin can delete any forum post" ON forum_posts;
CREATE POLICY "Admin can delete any forum post"
  ON forum_posts FOR DELETE
  TO authenticated
  USING (is_admin());

-- Forum Replies: admin pode deletar qualquer resposta
DROP POLICY IF EXISTS "Admin can delete any forum reply" ON forum_replies;
CREATE POLICY "Admin can delete any forum reply"
  ON forum_replies FOR DELETE
  TO authenticated
  USING (is_admin());
```

Cada política autoriza a operação `DELETE` sobre a respectiva tabela, para o papel `authenticated`, **condicionada** à função `is_admin()` retornar verdadeiro. O par `DROP POLICY IF EXISTS` seguido de `CREATE POLICY` torna a migração **idempotente** — pode ser reexecutada sem erro.

A migração ainda inclui, no cabeçalho, a documentação dos **pré-requisitos** (`migration_forum.sql` e `migration_admin_roles.sql`), boa prática que facilita a aplicação correta da migração por quem opera o banco.

6.3 Descoberta Relevante — Fechamento de uma Lacuna Funcional

A análise cruzada desta migração com o **código de frontend já existente** revela o ponto mais interessante desta entrega. O componente `src/pages/ForumPost.jsx` **já continha**, antes desta migração, a lógica de interface para exclusão administrativa:

```
const canDelete = isAuthor || isAdmin // botão de excluir post
const canDeleteReply = isReplyAuthor || isAdmin // botão de excluir resposta
```

Ou seja: a interface **já exibia os botões "Excluir"** para o administrador, e as funções `handleDeletePost` / `handleDeleteReply` **já executavam** as chamadas `supabase.from('forum_posts').delete()`. Entretanto, **sem as políticas RLS desta migração**, o banco de dados **silenciosamente rejeitava** essas exclusões: o Supabase, com RLS ativo, simplesmente **não exclui** as linhas para as quais nenhuma política de `DELETE` concede permissão — e o faz **sem retornar erro explícito** na operação padrão.

A consequência prática é que, **antes desta entrega**, havia um **defeito latente**: o administrador clicava em "Excluir", a interface aparentava sucesso, mas a postagem **permanecia no banco**. Esta migração, portanto, **não adiciona uma funcionalidade nova do zero — ela conserta uma funcionalidade que existia apenas pela metade**, alinhando a camada de persistência (RLS) à camada de apresentação (botões já presentes).

Esse achado é instrutivo sob dois ângulos:

1. **Reforça uma observação da primeira edição** sobre o tratamento inconsistente de erros: a operação de `delete` no frontend **não verifica o erro retornado** nem confere se alguma linha foi de fato afetada. Se verificasse, o defeito latente teria sido detectado mais cedo.
2. **Ilustra o risco de funcionalidades "meio implementadas"** — código de frontend e código de banco evoluindo em commits separados, com uma janela de inconsistência entre eles.

6.4 Análise de Segurança da Entrega 4

Aspecto	Avaliação	Comentário
Restrição da exclusão a administradores	✓ Correto	<code>USING (is_admin())</code> é avaliado no servidor
Idempotência da migração	✓ Correto	<code>DROP POLICY IF EXISTS</code> antes de <code>CREATE</code>
Escopo da permissão	✓ Adequado	Concede apenas <code>DELETE</code> ; não afeta <code>SELECT</code> / <code>INSERT</code> / <code>UPDATE</code>
Aplicação no servidor	✓ Correto	RLS é inviolável a partir do cliente
Confirmação no frontend	✓ Presente	<code>handleDeletePost</code> usa <code>confirm(...)</code> antes de excluir
Verificação do resultado da exclusão	⚠ Ausente	O frontend não checa o erro retornado pelo <code>delete()</code>

Do ponto de vista de **segurança**, a entrega é **correta e bem fechada**: a autorização ocorre no banco de dados, ponto inviolável a partir do cliente. Um usuário não administrador, ainda que manipule o frontend para forçar a exibição de um botão de exclusão, **não conseguirá** excluir conteúdo de terceiros, pois a política RLS o impedirá no servidor.

6.5 Recomendações Específicas da Entrega 4

1. **Adicionar verificação de resultado** às funções `handleDeletePost` e `handleDeleteReply` no frontend: checar o `error` retornado e confirmar que a linha foi efetivamente removida, exibindo feedback adequado.
 2. **Considerar exclusão lógica (soft delete)** em vez de exclusão física, para fins de auditoria — uma plataforma institucional pode precisar registrar **o que foi removido, por quem e quando**. Uma coluna `deleted_at` preservaria o histórico de moderação.
 3. **Avaliar a moderação por monitores**: atualmente apenas o administrador pode excluir conteúdo de terceiros. Dependendo do modelo operacional, pode fazer sentido estender (com nova política RLS) a capacidade de moderação aos monitores.
-

7. ENTREGA 5 — GERENCIAMENTO DINÂMICO DE OPÇÕES DE QUIZ

Commit: `e081751` — *"feat: add functionality to manage quiz options with add and remove buttons"* **Data:** 19 de maio de 2026 **Arquivos:** `src/pages/admin/AdminDisciplineEdit.jsx` (modificado), `src/pages/admin/AdminDisciplineEdit.css` (modificado)

7.1 Contexto e Motivação

A primeira edição descreveu `AdminDisciplineEdit` como o componente de **autoria de conteúdo** — a tela em que o administrador cria e edita disciplinas, aulas, materiais e questões de quiz. Até esta entrega, o formulário de criação de questões operava com um **número fixo de quatro alternativas** (A, B, C, D), conforme o estado inicial do formulário:

```
question: '', options: ['', '', '', ''], correct_option: 0, ...
```

A motivação da Entrega 5 é conferir **flexibilidade pedagógica** ao autor das questões: permitir que uma questão tenha uma **quinta alternativa opcional (E)**, e que essa alternativa possa ser removida caso o autor mude de ideia. É uma melhoria de **experiência do autor de conteúdo**.

7.2 As Funções `addQuizOption` e `removeQuizOption`

A entrega adiciona duas funções ao componente:

```
const addQuizOption = () => {
  setQuizForm(prev => ({ ...prev, options: [...prev.options, ''] }))
}

const removeQuizOption = (index) => {
  setQuizForm(prev => {
    const options = prev.options.filter((_, i) => i !== index)
    let correct_option = prev.correct_option
    if (correct_option === index) correct_option = 0
    else if (correct_option > index) correct_option -= 1
    return { ...prev, options, correct_option }
  })
}
```


A função `addQuizOption` é trivial: acrescenta uma string vazia ao array de opções, de forma **imutável** (espalhamento `[...prev.options, '']`), respeitando a regra fundamental do React de **não mutar o estado diretamente**.

A função `removeQuizOption`, embora curta, contém a **lógica mais sutil — e mais bem resolvida — desta entrega**: o reajuste do índice da resposta correta.

7.3 Análise da Lógica de Reindexação da Resposta Correta

O campo `correct_option` armazena o **índice** (posição) da alternativa correta dentro do array `options`. Ao remover uma alternativa, esse índice pode se tornar **inválido ou apontar para a alternativa errada**. A função trata os três cenários possíveis com correção:

Cenário	Condição	Tratamento	Justificativa
A opção removida era a correta	<code>correct_option === index</code>	<code>correct_option = 0</code>	A referência se perdeu; recai-se com segurança na alternativa A
A opção removida estava antes da correta	<code>correct_option > index</code>	<code>correct_option - = 1</code>	Todas as opções após a removida "deslizam" uma posição para trás
A opção removida estava depois da correta	<code>correct_option < index</code>	(nenhuma alteração)	A posição da correta não é afetada

Esta lógica está **inteiramente correta**. O cenário mais delicado — remover uma opção que vem **antes** da resposta correta — é tratado com o decremento `correct_option -= 1`, que mantém a resposta correta apontando para a alternativa certa após o "deslizamento" do array. Sem esse decremento, a remoção de uma opção anterior à correta faria o gabarito apontar silenciosamente para a alternativa errada — um **defeito grave de integridade de conteúdo**, que a implementação **previne corretamente**.

O fato de a função tratar explicitamente os três casos demonstra **maturidade na implementação** e atenção a casos de borda — exatamente o tipo de cuidado cuja ausência a primeira edição lamentava em outras partes do sistema.

7.4 Interface e Restrições

A interface aplica duas **restrições de limite** ao número de alternativas:

```

{/* botão de remover: só aparece a partir da 5ª opção */}
{i >= 4 && (
  <button className="btn-remove-option" onClick={() => removeQuizOption(i)} ...>
    <FiTrash2 />
  </button>
)}

{/* botão de adicionar: só aparece com menos de 5 opções */}
{quizForm.options.length < 5 && (
  <button className="btn-add-option" onClick={addQuizOption}>
    <FiPlus /> Adicionar Opção E (opcional)
  </button>
)}

```

As regras resultantes são:

- **Mínimo de 4 alternativas:** o botão de remover só é renderizado para opções de índice `>= 4` — ou seja, **apenas a 5ª opção (E) pode ser removida**. As alternativas A, B, C e D são **permanentes**, garantindo que toda questão tenha sempre, no mínimo, quatro alternativas.
- **Máximo de 5 alternativas:** o botão "Adicionar Opção E" só aparece quando há **menos de 5** opções, impedindo a criação de uma sexta.

O resultado é um intervalo controlado de **4 a 5 alternativas por questão** — decisão de design coerente com o formato pedagógico de questões de múltipla escolha e que **previne entradas degeneradas** (uma questão com 2 ou com 10 alternativas).

Os ícones reutilizam `FiPlus` e `FiTrash2`, **já importados** no componente — não houve necessidade de novas importações. O arquivo CSS recebeu 40 linhas para os estilos `.btn-add-option` (botão tracejado, em estilo "adicionar", na cor institucional) e `.btn-remove-option` (botão quadrado, em tom de alerta avermelhado), ambos com estados `:hover` — **consistentes com a linguagem visual** do restante da tela de edição.

7.5 Avaliação da Qualidade da Implementação

Pontos fortes:

- **Atualizações de estado imutáveis:** ambas as funções usam o padrão funcional `setQuizForm(prev => ...)` com espalhamento, em plena conformidade com as boas práticas do React.
- **Tratamento correto e completo de casos de borda** na reindexação da resposta correta.
- **Restrições de limite bem aplicadas**, prevenindo questões malformadas.
- **Reuso de ícones e consistência visual** com o restante do componente.
- **Escopo cirúrgico:** 71 linhas, nenhuma remoção, nenhum efeito colateral sobre o código existente — uma entrega **limpa e contida**.

Pontos de atenção:

- **Crescimento de um componente já grande:** `AdminDisciplineEdit.jsx` é um componente extenso e multifacetado (gerencia disciplinas, aulas, materiais e quizzes). A entrega é pequena, mas soma-se à massa de um componente que, como `DisciplineDetail` e `AdminReports`, é candidato a **decomposição futura** em subcomponentes.
- **Persistência do campo `options` como array:** convém confirmar que o backend e o schema da tabela de questões lidam corretamente com arrays de tamanho variável (4 ou 5) — a entrega assume essa flexibilidade, e o estado inicial do formulário continua sendo de 4 posições, o que é consistente.

7.6 Recomendações Específicas da Entrega 5

1. **Validar, no momento de salvar a questão**, que nenhuma alternativa exibida está vazia — especialmente a 5ª opção (E), que, sendo opcional, pode ser deixada em branco por engano.
 2. **Considerar a extração** da edição de quizzes de `AdminDisciplineEdit.jsx` para um subcomponente próprio, no contexto da recomendação geral de decomposição.
 3. **Cobrir `removeQuizOption` com testes unitários** — sua lógica de reindexação, embora correta, é o tipo de código sutil que se beneficia enormemente de testes de regressão (é um excelente candidato, ao lado de `accessLevels.js`, para inaugurar a suíte de testes).
-

8. ANÁLISE DE IMPACTO CONSOLIDADA

Este capítulo agrega os efeitos das cinco entregas sobre as dimensões de qualidade estabelecidas na primeira edição, oferecendo uma visão sistêmica do período.

8.1 Impacto sobre a Arquitetura

O período **preservou integralmente** a arquitetura em camadas descrita na primeira edição (Apresentação → Lógica de Negócio → Abstração de Dados → Persistência). Nenhuma entrega exigiu reestruturação arquitetural; **todas foram aditivas**.

A contribuição arquitetural mais significativa foi a **introdução de um segundo eixo de autorização** (níveis de acesso) pela Entrega 2. Esse eixo foi incorporado **respeitando a infraestrutura existente** — propagou-se pelo `AuthContext` (padrão Observer) e pela camada de domínio (`accessLevels.js`, módulo puro) sem violar fronteiras de contexto.

A criação do módulo `accessLevels.js` é, isoladamente, o **melhor exemplo de design do período**: módulo puro, coeso, com responsabilidade única, sem dependências e trivialmente testável. Recomenda-se que ele sirva de **referência de estilo** para futuros módulos de lógica de domínio.

Saldo arquitetural do período: positivo. A arquitetura demonstrou-se **extensível** e absorveu cinco entregas heterogêneas sem dívida estrutural — em conformidade com o Princípio Aberto/Fechado.

8.2 Impacto sobre a Segurança

Vetor de segurança	Estado na 1ª edição	Efeito do período	Estado atual
Exposição da chave do Gemini no bundle	✗ Vulnerabilidade crítica	Nenhuma alteração	✗ Permanece crítica
Controle de acesso (autorização)	✓ Bom (RLS + roles)	Reforçado (níveis + RLS de fórum)	✓ Aprimorado
Enforcement de troca de senha	✓ Mecanismo existente	Ativado para novos alunos	✓ Operacionalizado
Moderação de conteúdo do fórum	⚠ Defeito latente (RLS ausente)	Corrigido	✓ Corrigido
PII em repositório versionado	— (não havia)	Introduzida (~767 e-mails)	✗ Novo risco (LGPD)
Validação server-side de operações sensíveis	✓ Presente	Reforçada (RPC de nível de acesso)	✓ Aprimorado

Leitura consolidada: o período teve efeito **misto** sobre a segurança. Houve **avanços reais** — o reforço do controle de acesso, a correção do defeito latente de moderação do fórum e o uso consistente de validação server-side e de `SECURITY DEFINER` bem empregado. Por outro lado, **a vulnerabilidade crítica nº 2 da primeira edição** (chave de API do Gemini exposta no bundle de frontend) **não foi endereçada** e permanece como o **risco de segurança mais grave da plataforma**. Além disso, a Entrega 1 **introduziu um novo risco** — a presença de centenas de e-mails reais no histórico do Git, com implicações de conformidade com a LGPD.

8.3 Impacto sobre a Manutenibilidade e a Complexidade

A primeira edição apontou três componentes sobrecarregados: `DisciplineDetail` (20+ estados), `AdminReports` (função `getDisciplineMetrics` com `CC = 8`) e, implicitamente, `AdminDisciplineEdit`.

As entregas do período **adicionaram pequenas quantidades de lógica a todos os três**:

- `DisciplineDetail` recebeu a lógica das duas novas abas (Entrega 2);
- `AdminReports` recebeu a função `exportStudentsToExcel`, com dois laços aninhados (Entrega 3);
- `AdminDisciplineEdit` recebeu as funções de opção de quiz (Entrega 5).

Nenhum incremento foi grande isoladamente, mas o efeito cumulativo **reforça a recomendação de refatoração** desses três componentes — agora com **prioridade elevada**. O caminho recomendado é a **decomposição em subcomponentes** e a **extração de lógica para módulos utilitários testáveis** (como `accessLevels.js` exemplifica).

Contraponto positivo: a qualidade **interna** do código novo é, em geral, **superior** à média descrita na primeira edição. Destacam-se a atualização otimista com rollback (`handleAccessLevelChange`), o tratamento completo de casos de borda (`removeQuizOption`) e o reuso de lógica em vez de duplicação (`exportStudentsToExcel`). O período produziu **código novo de boa qualidade**, ainda que o tenha **acomodado em componentes que já eram grandes**.

8.4 Impacto sobre as Dependências

O período adicionou **uma única dependência de produção**: `xlsx@0.18.5`. A árvore de dependências permanece **enxuta** — uma das características positivas destacadas na primeira edição. A recomendação é monitorar avisos de segurança dessa biblioteca e considerar carregá-la sob demanda (ver seção 5.2 e 5.6).

8.5 Impacto sobre a Testabilidade

A testabilidade era o **índice mais baixo** da primeira edição (40/100), com **cobertura de testes de 0%**. O período **não adicionou nenhum teste automatizado** — a cobertura permanece em 0%.

Entretanto, o período **criou ótimas oportunidades** para iniciar uma suíte de testes:

- `accessLevels.js` — módulo **puro**, sem dependências; `hasAccessLevel` é testável com um punhado de asserções;
- `removeQuizOption` — lógica de reindexação determinística e de fácil cobertura;
- as funções de cálculo reaproveitadas pela exportação Excel.

Recomenda-se **fortemente** que o próximo ciclo de desenvolvimento introduza um *test runner* (Vitest, dada a base Vite) e escreva os primeiros testes justamente sobre esses alvos. Seria a primeira redução concreta do ponto crítico nº 1 da primeira edição.

8.6 Situação dos Pontos Críticos Herdados da Primeira Edição

#	Ponto crítico (1ª edição)	Situação após o período
1	Ausência de suíte de testes (cobertura 0%)	✗ Não alterado — permanece 0%
2	Exposição da chave de API do Gemini no bundle	✗ Não alterado — permanece crítico
3	Complexidade ciclomática elevada	⚠ Levemente agravado — incrementos em 3 componentes
4	Tratamento de erros inconsistente	⚠ Misto — melhora em <code>handleAccessLevelChange</code> ; lacunas em <code>exportStudentsToExcel</code> e nas exclusões do fórum
5	Ausência de TypeScript	✗ Não alterado — código novo permanece em JS sem tipagem
6	Ausência de sincronização em tempo real	✗ Não alterado — polling de 30s mantido

Síntese: dos seis pontos críticos herdados, **nenhum foi resolvido**, **um foi parcialmente mitigado** (nº 4, em um ponto específico), **um foi levemente agravado** (nº 3) e **quatro permanecem inalterados**. A plataforma evoluiu em **funcionalidade**, mas a **dívida técnica estrutural identificada na primeira edição segue, em sua maior parte, pendente**.

8.7 Reavaliação dos Índices de Qualidade ISO/IEC 25010

Com base na análise consolidada, os índices da primeira edição são reavaliados a seguir. As variações são **modestas e justificadas** — refletem que o período entregou valor funcional sem, contudo, atacar a dívida estrutural.

Dimensão	1ª edição	2ª edição	Variação	Justificativa
Manutenibilidade	70	69	▼ 1	Código novo bom, porém acomodado em componentes já grandes
Testabilidade	40	40	=	Sem testes adicionados; cobertura segue em 0%
Segurança	75	74	▼ 1	Avanços em autorização compensados pela PII no repositório
Desempenho	75	74	▼ 1	Exportação síncrona e bundle maior (<code>xlsx</code>)
Conformidade com SOLID	75	77	▲ 2	<code>accessLevels.js</code> e o uso de Open/Closed elevam a média
Conformidade com Clean Code	65	67	▲ 2	Código novo legível, com boa nomenclatura e funções coesas

Funcionalidade (ISO 25010): este é o índice que **mais avançou** no período — embora não quantificado numericamente na primeira edição (estimado em 90%). A plataforma ganhou controle de acesso por níveis, exportação de dados, moderação de fórum e melhor autoria de quizzes. **A maturidade funcional do produto cresceu de forma clara.**

Conclusão da reavaliação: o sistema encontra-se, hoje, **mais completo funcionalmente** e com **melhor qualidade de código nas partes novas**, porém **estruturalmente no mesmo patamar** da primeira edição. As pequenas variações negativas em Manutenibilidade, Segurança e Desempenho não indicam deterioração relevante — indicam que **o período priorizou entregar funcionalidades em vez de quitar dívida técnica**, decisão legítima para uma fase de operacionalização, mas que **não pode se repetir indefinidamente** sob pena de acúmulo de dívida.

9. CONCLUSÕES E RECOMENDAÇÕES

9.1 Resumo Executivo

Entre 6 de abril e 19 de maio de 2026, a plataforma **Capacita Portos** recebeu **cinco entregas de funcionalidade**, somando aproximadamente **2.077 linhas de código** distribuídas por **16 arquivos**. O período caracterizou-se pela **operacionalização do produto** — a plataforma transicionou para uso real, com uma turma concreta de alunos vinculados à autoridade portuária, e o esforço de desenvolvimento concentrou-se predominantemente nas **ferramentas de gestão e administração**.

As entregas foram, em geral, de **boa qualidade de implementação**. Destacam-se positivamente: o módulo `accessLevels.js` (design exemplar — puro, coeso, testável); o tratamento de atualização otimista com rollback em `AdminUsers`; o tratamento completo de casos de borda na reindexação de opções de quiz; e o reuso de lógica na exportação Excel. As migrações SQL revelaram-se, em sua maioria, **idempotentes, defensivas e bem documentadas**.

A entrega mais relevante do ponto de vista arquitetural foi o **Sistema de Níveis de Acesso**, que introduziu um segundo eixo de autorização sem violar a arquitetura existente. A entrega mais instrutiva foi a **Moderação do Fórum**, que revelou — e corrigiu — um defeito latente em que a interface prometia uma ação que o banco silenciosamente recusava.

Entretanto, o período **não atacou a dívida técnica estrutural** diagnosticada na primeira edição. Dos seis pontos críticos herdados, nenhum foi resolvido. Em especial, **a vulnerabilidade crítica de exposição da chave de API do Gemini permanece aberta**, e a Entrega 1 **introduziu um novo risco de conformidade** ao versionar centenas de e-mails reais no histórico do Git.

Veredito geral: o período foi **funcionalmente produtivo e tecnicamente competente nas partes novas**, mas **estruturalmente conservador**. A plataforma está mais útil, porém carrega a mesma dívida técnica de seis semanas atrás — agora um pouco maior.

9.2 Pontos Fortes do Período

1. **Arquitetura comprovadamente extensível** — cinco entregas absorvidas sem refatoração estrutural.
2. **Qualidade do código novo** — `accessLevels.js`, atualização otimista, tratamento de casos de borda.
3. **Migrações SQL maduras** — idempotentes, transacionais, defensivas e documentadas.

4. **Defesa em profundidade** — validação consistente no servidor (RPC + RLS), e não apenas no cliente.
5. **Correção de defeito latente** — a moderação do fórum agora funciona de ponta a ponta.
6. **Avanço claro da maturidade funcional** do produto.

9.3 Pontos de Atenção do Período

1. **(Crítico) PII no repositório** — ~767 e-mails reais embutidos no histórico do Git (risco LGPD).
2. **(Crítico, herdado) Chave do Gemini exposta no bundle** — não endereçada no período.
3. **(Débito técnico) Gating de conteúdo apenas no cliente** — exigirá RLS antes da publicação de conteúdo real nas abas de Reflexão e Artigo Técnico.
4. **Tratamento de erro ausente** em `exportStudentsToExcel` e na verificação do resultado das exclusões do fórum.
5. **Componentes grandes ficaram maiores** — `DisciplineDetail`, `AdminReports` e `AdminDisciplineEdit`.
6. **Cobertura de testes mantida em 0%** — apesar de o período ter criado alvos ideais para testar.
7. **Pequenas inconsistências** — duplicação de listas de níveis, RPC `get_my_access_level` não utilizada, CSS `.access-level-select` ausente.

9.4 Recomendações Prioritárias para o Próximo Ciclo

As recomendações abaixo estão ordenadas por prioridade. As prioridades **P0** devem ser tratadas **antes** de novas funcionalidades.

Prioridade	Recomendação	Origem
P0	Remover a chave de API do Gemini do bundle de frontend; mover a chamada ao LLM para um <i>backend proxy</i> (ex.: Supabase Edge Function)	Ponto crítico nº 2
P0	Tratar a PII versionada: mover dados para fora do VCS e avaliar, com o DPO, a limpeza do histórico do Git	Entrega 1
P0	Planejar e implementar a proteção server-side (RLS por <code>access_level</code>) do conteúdo das novas abas antes de publicar material real	Entrega 2
P1	Introduzir um <i>test runner</i> (Vitest) e escrever os primeiros testes sobre <code>accessLevels.js</code> e <code>removeQuizOption</code>	Ponto crítico nº 1
P1	Adicionar tratamento de erro a <code>exportStudentsToExcel</code> e verificação de resultado às exclusões do fórum	Ponto crítico nº 4
P1	Verificar a existência da política RLS de auto-leitura de <code>user_roles</code>	Entrega 2
P2	Refatorar <code>DisciplineDetail</code> , <code>AdminReports</code> e <code>AdminDisciplineEdit</code> em subcomponentes; extrair lógica para módulos utilitários	Ponto crítico nº 3
P2	Avaliar carregamento sob demanda da rota <code>AdminReports</code> e da biblioteca <code>xlsx</code>	Entrega 3
P2	Adicionar o conteúdo real às abas de Reflexão e Artigo Técnico (hoje em estado de andaime)	Entrega 2
P3	Corrigir as inconsistências menores: duplicação de listas de níveis, RPC não utilizada, CSS ausente	Entregas 2
P3	Avaliar a adoção incremental de TypeScript e de sincronização em tempo real (Supabase Realtime)	Pontos críticos nº 5 e 6
P3	Considerar <i>soft delete</i> e log de auditoria para a moderação do fórum	Entrega 4

9.5 Matriz de Riscos

Risco	Probabilidade	Impacto	Severidade	Mitigação
Abuso da chave do Gemini exposta	Média	Alto	Alta	Backend proxy (P0)
Vazamento/uso indevido da PII versionada	Baixa	Alto	Média-Alta	Expurgo e segregação (P0)
Acesso indevido a conteúdo das abas avançadas	Baixa (hoje)	Médio	Média (futura)	RLS por nível (P0, antes do conteúdo)
Regressão silenciosa por ausência de testes	Alta	Médio	Alta	Suíte de testes (P1)
Falha de exportação sem feedback ao usuário	Média	Baixo	Baixa-Média	<code>try/catch</code> (P1)
Erosão da manutenibilidade por componentes grandes	Alta	Médio	Média	Refatoração (P2)

9.6 Considerações Finais

A plataforma **Capacita Portos** encerra o período analisado como um produto **funcionalmente mais maduro, mais útil e operacionalmente real**. As cinco entregas foram bem executadas no nível da implementação e demonstram uma equipe capaz de escrever código de boa qualidade e migrações de banco de dados cuidadosas.

O desafio do próximo ciclo é **de natureza diferente do que foi o deste período**. Enquanto o intervalo de abril a maio foi corretamente dedicado a **entregar funcionalidades para operacionalizar o produto**, o ciclo seguinte deve, com igual disciplina, dedicar-se a **quitar a dívida técnica acumulada** — em particular as três prioridades **P0**: a chave de API exposta, a PII versionada e a proteção server-side do conteúdo por nível de acesso. Essas três pendências não são opcionais: a primeira é uma vulnerabilidade de segurança ativa, a segunda é uma exposição de conformidade legal, e a terceira é uma condição **prévia** para que a própria funcionalidade de níveis de acesso entregue valor real com segurança.

Recomenda-se que o ciclo seguinte seja **explicitamente equilibrado** entre funcionalidade e saúde estrutural, e que a **terceira edição** deste relatório, a ser emitida ao final desse ciclo, possa registrar — pela primeira vez desde o início desta série — a **redução efetiva** dos pontos críticos historicamente pendentes, em especial o **início de uma suíte de testes automatizados** e a **eliminação da vulnerabilidade da chave de API**.

10. PROJETO PARALELO EM DESENVOLVIMENTO: CAPACITA PORTOS INTERNO

Este capítulo, de natureza **prospectiva** e **declarativa**, apresenta um projeto paralelo em desenvolvimento, denominado **Capacita Portos Interno**. Diferentemente dos capítulos anteriores — que documentam código já entregue e versionado — este capítulo descreve uma **iniciativa em curso**, ainda não materializada em commits do repositório **Treinamento**, mas que **compartilhará integralmente** o repertório de funcionalidades hoje presente na plataforma analisada nas seções precedentes.

A inclusão desta seção neste relatório justifica-se por três razões: (1) consolida, em um único documento técnico, o **portfólio atual e previsto** de soluções de e-learning vinculadas ao ecossistema Capacita Portos; (2) permite que a equipe técnica e os tomadores de decisão tenham, desde já, uma **referência detalhada** das funcionalidades a serem replicadas, antes mesmo da abertura do novo repositório; e (3) registra, com clareza, **o ponto de partida funcional** do novo projeto, que é precisamente o estado de maturidade descrito nos capítulos 1 a 9.

10.1 Apresentação Geral e Justificativa Estratégica

O **Capacita Portos Interno** é uma **plataforma de e-learning gamificada irmã** da plataforma Capacita Portos atualmente em operação, com a qual compartilhará **arquitetura, padrões de projeto, paleta de funcionalidades e linguagem visual**, diferenciando-se exclusivamente por seu **público-alvo, conteúdo pedagógico e nível de profundidade técnica**. Enquanto a plataforma atual é orientada à **capacitação ampla** do ecossistema portuário (incluindo público externo, parceiros, prestadores de serviço e demais agentes da cadeia logística), o Capacita Portos Interno terá como objetivo a **qualificação contínua, técnica e operacional do quadro próprio da autoridade portuária** — servidores, analistas, técnicos, engenheiros, operadores, gestores e equipes finalísticas que atuam diretamente nas operações, na fiscalização, na manutenção e na administração do complexo portuário.

A justificativa estratégica para o desenvolvimento de uma **plataforma interna dedicada** — em vez de simplesmente abrir uma trilha interna dentro da plataforma atual — apoia-se em quatro pilares:

1. **Especialização do conteúdo pedagógico:** o material destinado ao público interno tende a ser **mais técnico, mais denso e mais regulatório**, abordando temas que pressupõem conhecimento prévio do domínio portuário (operações de cais, normas técnicas,

procedimentos administrativos, instruções normativas internas). Misturar esse conteúdo com a oferta voltada ao público externo poderia gerar ruído pedagógico em ambas as direções: o conteúdo interno apareceria como excessivamente técnico para o público externo, e o conteúdo externo apareceria como básico e irrelevante para o servidor experiente.

2. **Segregação de dados sensíveis:** o conteúdo interno pode envolver **procedimentos operacionais sensíveis**, regras internas de fiscalização, instruções normativas restritas e, eventualmente, dados de desempenho funcional dos próprios servidores. Manter esse conteúdo em uma plataforma fisicamente segregada da plataforma pública reduz a superfície de exposição e facilita a aplicação de controles mais rígidos de acesso, auditoria e retenção.
3. **Identidade e narrativa institucional:** uma plataforma interna comunica, simbolicamente, o **valor que a organização atribui à capacitação do seu próprio quadro**. Investir em uma plataforma dedicada — com identidade visual, terminologia e abordagem própria — reforça a cultura de aprendizagem contínua e diferencia o esforço de capacitação interna do esforço de capacitação externa, evitando que o servidor perceba o conteúdo interno como uma versão "secundária" do que é oferecido ao público em geral.
4. **Liberdade de evolução técnica:** ao manter duas bases de código irmãs porém independentes, cada plataforma pode evoluir em **ritmo, cadência e prioridades próprias**, sem que uma decisão de uma comprometa a estabilidade da outra. Funcionalidades experimentais podem ser testadas no Capacita Portos Interno (com público mais conhecido e tolerante a *bugs*) antes de migrarem para a plataforma pública; inversamente, ajustes operacionais críticos podem ser feitos na plataforma pública sem interferir na cadência da plataforma interna.

10.2 Público-Alvo: O Servidor e o Quadro Técnico Portuário

O **público-alvo primário** do Capacita Portos Interno é o **quadro funcional da autoridade portuária**, considerado em sua diversidade de funções e níveis técnicos. Esse universo inclui, sem se restringir aos seguintes perfis:

- **Servidores administrativos**, atuantes em áreas-meio (recursos humanos, finanças, suprimentos, jurídico, planejamento, comunicação institucional, tecnologia da informação);
- **Analistas e técnicos finalísticos**, atuantes nas áreas-fim (operações portuárias, infraestrutura, meio ambiente, segurança portuária, regulação, fiscalização, projetos);
- **Engenheiros** (civis, mecânicos, elétricos, ambientais, de segurança do trabalho), envolvidos em obras, manutenção, dragagem, sinalização náutica e projetos de infraestrutura;
- **Operadores e supervisores de operação**, vinculados à movimentação de cargas, controle de acessos, autorização de manobras, dragagem e operações com cabotagem e longo curso;

- **Equipes de segurança portuária**, no escopo do **Código ISPS** (International Ship and Port Facility Security Code), incluindo agentes de segurança da instalação portuária (AFPF), oficiais de proteção e equipes de monitoramento e controle de acesso;
- **Gestores de nível tático e estratégico**, responsáveis pela coordenação de áreas, contratos, convênios e relações institucionais;
- **Equipes de auditoria e controle interno**, com atribuições de verificação de conformidade regulatória, financeira e operacional;
- **Estagiários e novos servidores em fase de ambientação**, para os quais a plataforma poderá funcionar como instrumento estruturado de **integração funcional (onboarding)**;
- **Conselheiros, dirigentes e demais membros de órgãos colegiados**, para os quais determinados conteúdos podem ser ofertados em formato de cápsulas executivas.

A diversidade de perfis demanda uma **estratificação cuidadosa de conteúdos** — viabilizada justamente pelo sistema de **níveis de acesso** já implementado na plataforma atual (Entrega 2 do período analisado neste relatório). O Capacita Portos Interno **herdará e potencializará** esse mecanismo, possivelmente expandindo-o ou reconfigurando seus níveis para refletir hierarquias funcionais, áreas de atuação ou exigências de habilitação específicas do quadro interno.

10.3 Diferenciação em Relação à Plataforma Atual

Embora arquitetura e funcionalidades sejam, em essência, **as mesmas** da plataforma atual, há quatro eixos sistemáticos de **diferenciação** entre a plataforma pública e a plataforma interna:

Eixo	Capacita Portos (atual)	Capacita Portos Interno (em desenvolvimento)
Público	Externo, parceiros, prestadores, cadeia logística ampliada	Quadro próprio da autoridade portuária
Conteúdo	Capacitação geral, introdutória e operacional	Aprofundamento técnico, regulatório e procedimental
Linguagem	Acessível, próxima da comunicação institucional pública	Técnica, regulatória, com vocabulário do domínio
Identidade	Marca institucional voltada ao ecossistema externo	Marca institucional voltada à valorização do quadro interno
Governança de acesso	Cadastro aberto, com curadoria	Cadastro fechado , atrelado à matrícula funcional
Profundidade dos quizzes	Aferição de conceitos gerais	Aferição de procedimentos, normas e estudos de caso
Materiais complementares	Cartilhas, vídeos institucionais	Instruções normativas internas, procedimentos operacionais padrão, anexos técnicos
Métricas administrativas	Engajamento, conclusão geral	Engajamento por unidade funcional, área e cargo
Conformidade	Boas práticas de privacidade e LGPD	Mesmas práticas acrescidas de tratamento como dado funcional, com requisitos adicionais de auditoria

Importante destacar: **a base de código é a mesma família**, mas o **público, o conteúdo e a curadoria são integralmente próprios**. A separação não é cosmética — é arquitetural, organizacional e pedagógica.

10.4 Arquitetura Prevista (Espelhada da Plataforma Atual)

A arquitetura do Capacita Portos Interno **espelhará integralmente** a arquitetura descrita na primeira edição deste relatório (capítulo 1) e mantida ao longo das entregas analisadas na presente edição (capítulos 3 a 7). Os pilares arquiteturais previstos são:

- **Camada de Apresentação:** SPA (Single Page Application) em **React 19** com **Vite**, replicando os componentes e páginas hoje existentes (Dashboard, DisciplineDetail, Forum, MyDoubts, AdminDisciplines, AdminReports, MonitorStudents, MonitorStudentDetail, entre outros);

- **Camada de Lógica de Negócio:** mantida no frontend via hooks e contextos (`AuthContext` para autenticação/autorização; módulos puros como `accessLevels.js` e `badges.js` para regras de domínio);
- **Camada de Abstração de Dados:** cliente único do **Supabase** (padrão Singleton já identificado na primeira edição), com chamadas tipadas e operações via PostgREST e RPC;
- **Camada de Persistência: PostgreSQL gerenciado pelo Supabase**, com **Row Level Security (RLS)** como mecanismo central de autorização, espelhando o conjunto de tabelas e migrações já existente — incluindo `user_roles` (com o campo `access_level`), `disciplines`, `lessons`, `materials`, `lesson_progress`, `user_progress`, `quiz_results`, `lesson_quiz_results`, `forum_posts`, `forum_replies`, `doubts`, `doubt_responses`;
- **Camada de Inteligência Artificial:** integração com o **Google Generative AI (Gemini)**, com **system prompt contextualizado por disciplina** — mas com a recomendação **incorporada desde o início de não expor a chave de API no bundle** (o débito P0 herdado da plataforma atual deverá ser **resolvido nativamente** na arquitetura do projeto interno, via *backend proxy* em Supabase Edge Functions);
- **Camada de Roteamento: React Router DOM v7**, com a mesma topologia de rotas protegidas, rotas administrativas e rotas de monitor, encapsuladas por `ProtectedRoute`, `AdminRoute` e `MonitorRoute` (padrão Decorator já catalogado).

Os mesmos **sete padrões de projeto** identificados na primeira edição (Observer, Strategy, Adapter/Wrapper, Singleton, Factory, Decorator e Event Emitter/Pub-Sub) **serão replicados** no Capacita Portos Interno, com **oportunidade de aprimoramento** pontual à luz das lições aprendidas:

- O padrão **Pub-Sub** poderá, desde a concepção, ser implementado via **Supabase Realtime**, substituindo o **polling de 30 segundos** apontado como ponto crítico nº 6 na primeira edição. Esta é uma oportunidade explícita de **não herdar** uma dívida técnica conhecida.
- A **complexidade ciclomática** dos componentes-chave (`DisciplineDetail`, `AdminReports`, `AdminDisciplineEdit`) poderá ser **endereçada na origem**, evitando a sobrecarga registrada nas seções 3.1.3 e 3.1.4 da primeira edição e nas seções 4.5, 5.4 e 7.5 desta segunda edição.
- A **suíte de testes**, ausente na plataforma atual (ponto crítico nº 1), poderá ser **incorporada desde o primeiro commit** do novo projeto, evitando que a cobertura inicie em 0%.
- A **tipagem estática (TypeScript)**, ausente na plataforma atual (ponto crítico nº 5), poderá ser **adotada desde o início** do Capacita Portos Interno, oferecendo type-safety por padrão em todos os contratos da aplicação.

Em outras palavras: o Capacita Portos Interno tem a **oportunidade arquitetural** de nascer **já corrigido** dos seis pontos críticos historicamente apontados para a plataforma atual. Recomenda-se que essa oportunidade seja explorada com **disciplina e intencionalidade**.

10.5 Catálogo Detalhado de Funcionalidades

A seguir, apresenta-se o **catálogo exaustivo de funcionalidades** previstas para o Capacita Portos Interno. Cada item descreve a funcionalidade em sua **forma já existente** na plataforma atual e, em seguida, detalha sua **especialização para o público interno e técnico**. A lista é deliberadamente extensa — sua função é constituir, neste documento, uma **referência funcional completa** sobre a qual o desenvolvimento do novo projeto poderá se apoiar.

10.5.1 Autenticação Institucional

Funcionalidade-base (herdada da plataforma atual): sistema de cadastro, login, recuperação de senha por e-mail e redefinição de senha, com autenticação via Supabase Auth, emissão de JWT e gerenciamento de sessão; estado global de autenticação propagado por `AuthContext` (padrão Observer) e acesso protegido a rotas via `ProtectedRoute`.

Especialização para o público interno: o módulo de autenticação do Capacita Portos Interno será **fechado e nominal**: o acesso será restrito a usuários previamente cadastrados pela administração, vinculados à sua **matrícula funcional** e ao seu **e-mail institucional** (`@portosrio.gov.br` ou domínio equivalente da autoridade portuária). Esta especialização traz consigo um conjunto de adaptações:

- **Cadastro inexistente para o usuário final:** diferentemente da plataforma atual, **não haverá tela pública de cadastro**. As contas serão criadas exclusivamente pela administração, em fluxo análogo ao já existente na tela `AdminUsers` da plataforma atual e nos scripts de inserção em massa documentados no capítulo 3 desta edição.
- **Domínio de e-mail restringido:** o servidor de autenticação validará, no momento do cadastro administrativo, que o e-mail informado pertence ao **domínio institucional** autorizado. Tentativas de cadastrar contas com e-mails externos resultarão em recusa.
- **Senhas provisórias com troca obrigatória no primeiro acesso:** o mecanismo `must_reset_password` já implementado e operacionalizado na plataforma atual (Entrega 1 deste período) será adotado **por padrão** para toda criação de conta no Capacita Portos Interno.
- **Política de senha reforçada:** comprimento mínimo, exigência de combinação de caracteres, vedação a senhas comuns e expiração periódica poderão ser configurados no Supabase Auth, em conformidade com a política de segurança da informação da organização.
- **Eventual integração com diretório institucional:** em fase posterior, poderá ser avaliada a integração com **Single Sign-On (SSO)** baseado no diretório de identidades da autoridade portuária (por exemplo, via OIDC ou SAML), eliminando a necessidade de credenciais específicas para a plataforma e centralizando ciclo de vida (criação/desativação) com o RH da organização.

- **Bloqueio automático ao desligamento:** quando integrado ao diretório institucional, a desativação da conta no diretório (ex.: em caso de desligamento do servidor) propagaria automaticamente para a plataforma, impedindo acesso por ex-servidores.

10.5.2 Trilha de Disciplinas Sequenciais

Funcionalidade-base: o aluno percorre uma **trilha ordenada de disciplinas**; a próxima disciplina só é desbloqueada após a conclusão da anterior. A ordenação é controlada pelo campo `order_index` da tabela `disciplines`, e o desbloqueio é determinado pelo conjunto de registros em `user_progress` com `completed = true`.

Especialização para o público interno: a estrutura de trilhas sequenciais será mantida, porém **organizada segundo eixos funcionais e técnicos** próprios da operação portuária. Trilhas previstas (ilustrativas, sujeitas à curadoria pedagógica final):

- **Trilha de Integração Funcional:** voltada a novos servidores, abrangendo missão institucional, estrutura organizacional, código de conduta, regimento interno, fluxos de processo administrativo e ambientação à autoridade portuária;
- **Trilha de Segurança Portuária:** abordando o Código ISPS, controle de acesso, gerenciamento de risco, procedimentos de emergência, exercícios e simulados, equipamentos de proteção individual e coletiva;
- **Trilha de Operações Portuárias:** englobando movimentação de carga, operações de cais, manuseio de cargas perigosas (IMDG), sinalização náutica, dragagem, controle de manobras, autorização de operação;
- **Trilha Ambiental:** licenciamento, gestão de resíduos, controle de efluentes, gestão da qualidade da água, monitoramento de biodiversidade, plano de emergência ambiental;
- **Trilha Regulatória:** marco regulatório do setor (Lei dos Portos, normas ANTAQ, instruções normativas da SEP/MPor, regulamento de exploração), conformidade contratual e fiscalização de arrendamentos;
- **Trilha Administrativa:** processo administrativo, licitações e contratos, gestão patrimonial, gestão orçamentária, prestação de contas, controle interno;
- **Trilha Técnica de Engenharia:** infraestrutura portuária, manutenção de equipamentos, sinalização, dragagem, obras civis, instalações elétricas e mecânicas;
- **Trilha de Tecnologia da Informação:** segurança da informação, governança de dados, sistemas internos, política de uso aceitável, LGPD aplicada à operação portuária.

A **sequencialidade** será preservada **dentro de cada trilha**, mas o usuário poderá ter **acesso simultâneo a múltiplas trilhas** conforme seu cargo, função ou área de atuação. Esse refinamento — múltiplas trilhas concorrentes — poderá demandar uma extensão do modelo de progresso existente, eventualmente introduzindo uma noção de "trilha" como agrupador de disciplinas, e relacionando o usuário às trilhas pertinentes a seu perfil.

10.5.3 Aulas em Vídeo

Funcionalidade-base: cada disciplina possui uma sequência ordenada de **aulas em vídeo**. O sistema suporta múltiplos provedores via a função `getEmbedUrl()` identificada na primeira edição (Padrão Factory), abrangendo YouTube, Vimeo, Google Drive e URLs genéricas. As aulas são exibidas em player embutido na página `DisciplineDetail`, com indicação visual de aula em andamento e de aula concluída.

Especialização para o público interno: a infraestrutura de vídeo será replicada, com as seguintes adaptações:

- **Hospedagem de vídeos institucionais não públicos:** parte significativa do conteúdo interno poderá envolver **vídeos não destinados ao público externo** (gravações de aulas com servidores expondo procedimentos internos, simulações operacionais, gravações de equipamentos em uso). Para esses casos, recomenda-se evitar provedores públicos (YouTube em modo público) e priorizar **hospedagem em ambiente controlado** — seja por canais com permissão restrita, vídeos não listados, Vimeo com proteção por domínio, ou armazenamento direto em **Supabase Storage** com URLs assinadas de duração limitada.
- **Reforço à `getEmbedUrl()`** : a função Factory poderá ser estendida para reconhecer URLs do Supabase Storage e gerar players com **token assinado** de acesso, garantindo que apenas usuários autenticados consigam reproduzir o vídeo.
- **Marcação de aula concluída condicionada a tempo de visualização:** o aviso atualmente exibido ("watch notice", funcionalidade mencionada nos commits históricos) poderá ser **reforçado** com a exigência de que o aluno permaneça com o vídeo aberto por uma proporção mínima de sua duração, antes de habilitar o botão de "marcar como concluída". Para o conteúdo interno — frequentemente regulatório e procedimental — essa garantia mínima de exposição é desejável.
- **Transcrição e legenda:** sempre que possível, vídeos serão acompanhados de **transcrição textual** e **legenda fechada**, atendendo a critérios de acessibilidade exigidos da Administração Pública.

10.5.4 Materiais Complementares

Funcionalidade-base: cada disciplina pode disponibilizar **materiais complementares** para download — PDFs, apresentações, planilhas. A funcionalidade de upload e armazenamento já está implementada (commit histórico `7bb4c7a`, com validação e política de armazenamento em Supabase Storage). Os materiais aparecem em uma aba dedicada na `DisciplineDetail`, com botão de download para cada arquivo.

Especialização para o público interno: os materiais complementares do Capacita Portos Interno tendem a ser **mais densos, mais técnicos e mais regulatórios** que os da plataforma pública. Tipologias previstas:

- **Instruções Normativas internas** e regulamentos da autoridade portuária;

- **Procedimentos Operacionais Padrão (POPs)** das diversas áreas funcionais;
- **Manuais técnicos** de operação, segurança e manutenção;
- **Anexos regulatórios** (resoluções ANTAQ, normas da Marinha, normas ambientais);
- **Estudos de caso** internos, com aprendizados de incidentes e boas práticas consolidadas;
- **Modelos e formulários** de uso recorrente (autorizações, registros, checklists);
- **Apresentações de capacitação** previamente ministradas em formato presencial;
- **Glossários técnicos** do domínio portuário e regulatório.

Recomenda-se que a página de materiais permita **busca textual** (por título e descrição) e **filtros por tipo de documento**, considerando que o volume de materiais internos tende a crescer substancialmente.

Aspectos de **segurança no armazenamento** dos materiais merecem atenção elevada no contexto interno:

- **Políticas RLS no Supabase Storage** devem restringir o download a usuários autenticados e, quando aplicável, ao nível de acesso adequado;
- **Marca-d'água personalizada** (com o nome do servidor que está baixando o arquivo) poderá ser aplicada a PDFs sensíveis, dificultando o vazamento e permitindo a rastreabilidade do redistribuidor original;
- **Versionamento de documentos** poderá ser incorporado, garantindo que o aluno consulte sempre a versão vigente do material e que versões antigas permaneçam acessíveis para auditoria.

10.5.5 Quiz por Aula

Funcionalidade-base: cada aula pode ter um **quiz individual** (Lesson Quiz), com perguntas de múltipla escolha, gabarito automático, comentário de correção (campo `correction_comment`) e armazenamento do resultado em `lesson_quiz_results`. A Entrega 5 deste período flexibilizou o número de alternativas para 4 ou 5 (capítulo 7).

Especialização para o público interno: o quiz por aula no Capacita Portos Interno servirá menos para "fixação de conceitos básicos" e mais para **aferição da compreensão técnica e procedimental** do servidor. Adaptações previstas:

- **Questões baseadas em situações reais:** os enunciados poderão descrever **estudos de caso** ou **situações operacionais** típicas, exigindo do aluno aplicação de norma ou procedimento, em vez de simples recuperação memorística;
- **Comentários de correção mais elaborados:** o campo `correction_comment` poderá ser explorado de forma intensiva, oferecendo, em cada alternativa, **justificativa pedagógica completa** — por que tal opção está correta ou por que está incorreta, com referência à norma, ao procedimento ou ao manual de origem;

- **Banco de questões com referência regulatória:** cada questão poderá manter, em metadado, **referência ao instrumento normativo** (artigo, lei, resolução, IN) que a fundamenta, facilitando a manutenção e a atualização do banco quando a norma muda;
- **Limite ampliado de alternativas (eventual):** embora o limite atual seja de 4 ou 5 alternativas, casos específicos de aferição (por exemplo, "marque todas as alternativas verdadeiras") podem demandar uma **modalidade adicional** de questão — funcionalidade não existente hoje, mas que poderia ser introduzida no projeto interno e, se bem-sucedida, eventualmente portada para a plataforma pública.

10.5.6 Quiz Final da Disciplina

Funcionalidade-base: ao final de cada disciplina, após a conclusão das aulas, é apresentado um **quiz final** mais abrangente. A nota mínima de aprovação é de **70%** (regra constante de `computeDisciplineBadges` e da lógica em `Quiz.jsx`). O resultado é registrado em `quiz_results`, e a aprovação dispara a marcação `user_progress.completed = true`. A funcionalidade de **auto-conclusão** para disciplinas sem quiz final (commit `f5b50f3`) também está incorporada.

Especialização para o público interno: o quiz final ganhará natureza de **aferição certificatória**. Adaptações:

- **Geração de certificado:** ao aprovar o quiz final de uma disciplina, o servidor poderá baixar um **certificado de conclusão** em PDF, contendo seu nome, matrícula funcional, disciplina, carga horária estimada, data de conclusão, percentual de acertos e um identificador único de verificação. Essa funcionalidade pode reaproveitar a infraestrutura de geração de relatórios (lib `xlsx` já incorporada e potencial nova lib para PDFs);
- **Banco de questões randomizado:** para reduzir o risco de circulação do gabarito entre servidores, o quiz final poderá selecionar, a cada tentativa, um **subconjunto aleatório** de um banco maior, garantindo que duas tentativas distintas não tenham, exatamente, as mesmas questões;
- **Limite de tentativas e cool-down:** dependendo do regime adotado pela área de capacitação, o quiz final poderá ter **número limitado de tentativas** ou **intervalo mínimo entre tentativas** (cool-down), em coerência com o caráter certificatório;
- **Registro funcional do desempenho:** o resultado da aprovação poderá ser exportado, mediante autorização, para o sistema de **gestão funcional** da autoridade portuária, integrando-se à trilha de desenvolvimento individual do servidor.

10.5.7 Sistema de Badges e Gamificação

Funcionalidade-base: o sistema de badges descrito em detalhe na primeira edição (Pattern Strategy, módulo `badges.js`) reconhece sete tipos de conquistas — `lesson_complete`, `lesson_quiz_done`, `lesson_quiz_perfect`, `all_lessons_complete`, `final_quiz_complete`, `discipline_complete` e `all_disciplines_complete` — com graduação em bronze, prata, ouro e diamante. O cálculo é determinístico, dado o conjunto de progresso do usuário.

Especialização para o público interno: a gamificação será mantida, porém **calibrada para o público profissional adulto**. Considerações:

- **Linguagem das conquistas:** em vez de denominações lúdicas, as conquistas poderão ser nomeadas de forma **institucional**, refletindo competências profissionais ("Especialista em Operações", "Capacitado em Segurança Portuária", "Apto em Conformidade Ambiental"). A estética poderá ser mais sóbria que a da plataforma pública;
- **Conquistas atreladas a competências:** a estrutura `BADGE_DEFS` poderá ser expandida para suportar **conquistas por área de competência**, não apenas por disciplina, permitindo que o conjunto de badges de um servidor expresse, em síntese visual, suas **áreas de capacitação certificada**;
- **Reconhecimento institucional:** o painel administrativo poderá oferecer relatórios de **servidores com conquistas notáveis**, que sirvam à área de gestão de pessoas como insumo para programas de reconhecimento, sucessão ou alocação;
- **Vinculação a desenvolvimento de carreira:** dependendo do desenho institucional, conquistas poderão integrar — como evidência — programas de **trilhas de carreira** ou de **progressão funcional**.

10.5.8 Ranking e Discipline Ranking

Funcionalidade-base: o sistema mantém um **ranking global** e um **ranking por disciplina** (commit histórico `f48deb9`), com base na pontuação consolidada de badges (`getAllDisciplineBadges`).

Especialização para o público interno: a competitividade entre servidores é um **terreno mais delicado** que a competitividade entre alunos externos. Recomendações:

- **Modo opt-in:** o ranking individual público poderá ser **opcional**, permitindo ao servidor escolher se deseja ou não aparecer em quadros visíveis a colegas. Essa decisão respeita autonomia e evita constrangimentos hierárquicos;
- **Ranking por unidade ou área:** alternativamente, o ranking poderá ser apresentado em granularidade **coletiva** — desempenho por área, gerência ou unidade funcional —, evitando exposição individual e estimulando dinâmica de equipe;
- **Ranking restrito à própria visão do servidor:** outra alternativa é manter o ranking, mas torná-lo **visível apenas ao próprio servidor**, como um instrumento de autoavaliação, sem exposição lateral;

- **Calibragem para mitigar viés de oportunidade:** servidores em férias, licença ou em funções com menor disponibilidade de tempo para capacitação não devem ser penalizados publicamente. O algoritmo de ranqueamento poderá considerar **janelas de atividade** ou normalização por tempo de exposição.

10.5.9 Fórum

Funcionalidade-base: o fórum (`forum_posts` e `forum_replies`) é um espaço de discussão **aberto** entre todos os usuários autenticados da plataforma, com categorias, autor identificado, contagem de respostas e ordenação por data. A moderação administrativa foi introduzida na Entrega 4 deste período (capítulo 6).

Especialização para o público interno: o fórum interno terá natureza de **comunidade de prática profissional**. Adaptações previstas:

- **Categorias alinhadas às áreas de atuação:** as categorias do fórum refletirão o organograma funcional ou as áreas técnicas (Operações, Segurança, Ambiental, Administrativo, TI, Engenharia), facilitando a localização de tópicos pertinentes;
- **Regras de uso explícitas:** o fórum interno exibirá, na sua página de entrada, um **regulamento de conduta** alinhado ao **código de ética** da organização, lembrando aos servidores que se trata de espaço institucional;
- **Visibilidade do cargo:** ao lado do nome do autor de um post, poderá ser exibida a **área ou cargo funcional**, conferindo contexto à discussão (por exemplo, "Analista — Operações Portuárias");
- **Moderação reforçada:** a infraestrutura de moderação introduzida pela Entrega 4 (políticas RLS de exclusão pelo admin) será **acrescida**, possivelmente, da capacidade de **moderação por monitores** (servidores designados pela área de capacitação), eventualmente com **fluxo de denúncia** de conteúdo inadequado;
- **Trilhas privadas dentro do fórum:** discussões classificadas pelo conteúdo poderão ter **visibilidade restrita** por nível de acesso ou por área, evitando que tópicos de natureza específica fiquem expostos a todo o quadro;
- **Soft delete e auditoria:** conforme recomendado no capítulo 6, recomenda-se que o fórum interno adote **exclusão lógica** (campo `deleted_at`) e **log de moderação**, registrando quem moderou o quê e quando, para fins de transparência.

10.5.10 Minhas Dúvidas e Sistema de Monitoria

Funcionalidade-base: os alunos podem enviar **dúvidas** individuais (tabela `doubts`), respondidas por **monitores** vinculados a disciplinas (tabela `doubt_responses`). O painel `MyDoubts` lista as dúvidas do aluno e o painel do monitor lista as dúvidas atribuídas. O `Layout` mantém um **badge de notificação** com polling a 30 segundos para indicar dúvidas com novas respostas.

Especialização para o público interno: a monitoria do Capacita Portos Interno terá natureza de **mentoria técnica intra-organizacional**. Adaptações:

- **Monitores são servidores designados:** os monitores serão **servidores mais experientes ou especialistas** da própria autoridade portuária, designados pela área de capacitação como **multiplicadores** em suas respectivas áreas de competência. A função se torna mecanismo de **transferência interna de conhecimento**;
- **Atribuição por área de competência:** o vínculo monitor-aluno será orientado pela **especialidade do monitor** e pela **trilha do aluno**, em vez de uma associação aluno-por-aluno como na plataforma atual;
- **Substituição do polling por Supabase Realtime:** conforme já recomendado, o badge de notificação de dúvidas poderá ser implementado, no projeto interno, via **subscription em tempo real**, eliminando o atraso de até 30 segundos e o ponto crítico nº 6 da primeira edição;
- **Privacidade da dúvida:** o conteúdo de uma dúvida individual de servidor pode envolver informação sensível sobre procedimento interno; recomenda-se que dúvidas e respostas tenham, por padrão, visibilidade **restrita ao par aluno-monitor** e, somente em escalações, aos administradores.

10.5.11 Chat com Inteligência Artificial (Google Gemini)

Funcionalidade-base: componente **AIChat** integrado ao Google Generative AI (Gemini), com **system prompt contextualizado por disciplina**, oferecendo assistente conversacional para tirar dúvidas. A primeira edição apontou, como vulnerabilidade crítica, a exposição da chave de API (**VITE_GEMINI_API_KEY**) no bundle de frontend — ponto **não corrigido** durante o período coberto por esta segunda edição.

Especialização para o público interno: o uso de IA generativa no contexto institucional interno demanda **rigor adicional**. Diretrizes específicas:

- **Backend proxy desde o nascimento:** o Capacita Portos Interno **não poderá**, em nenhuma circunstância, embarcar chaves de API de provedores externos no bundle de frontend. A chamada ao Gemini (ou a qualquer LLM equivalente) ocorrerá **exclusivamente** por meio de uma **Supabase Edge Function** (ou serviço equivalente), que armazena a chave de forma segura no ambiente do servidor e expõe ao frontend apenas um endpoint autenticado. Esta é uma **exigência arquitetural mandatória**, herdada como aprendizado da plataforma atual;
- **System prompt institucional:** o prompt de sistema da IA será **fortemente contextualizado**, incluindo a missão da autoridade portuária, restrição de domínio (a IA orienta sobre temas da disciplina e não responde sobre assuntos não relacionados), tom institucional e instrução para **não revelar nem inferir** dados pessoais de servidores;
- **Política de não retenção de dados:** o uso da IA deverá ser configurado, sempre que o provedor permitir, em **modo sem retenção** — ou seja, sem que o conteúdo das conversas

seja usado pelo provedor para treinamento. Isso protege informações operacionais e procedimentais discutidas com o assistente;

- **Avaliação contínua de respostas:** o assistente poderá registrar **feedback do servidor** (útil/não útil, correção sugerida), gerando insumo para curadoria contínua do prompt e do conteúdo associado;
- **Limites de uso:** quotas por usuário e período poderão ser aplicadas, dado que o custo por inferência é real e o orçamento institucional é finito;
- **Avaliação periódica do provedor:** o vínculo a um provedor único (Google Gemini) será revisto periodicamente. A arquitetura preverá **abstração** suficiente para que o motor de IA possa ser substituído (por outro modelo do mesmo provedor ou por outro provedor) **sem refatoração extensa** do código consumidor.

10.5.12 Painel do Monitor

Funcionalidade-base: os monitores acessam um conjunto dedicado de páginas — `MonitorStudents`, `MonitorStudentDetail`, `MonitorDoubts`, `MonitorDoubtDetail` — para acompanhar o progresso de seus alunos, visualizar último acesso, responder dúvidas e marcar interações.

Especialização para o público interno: o painel do monitor no Capacita Portos Interno será **mais analítico**, oferecendo:

- **Visão por área de competência:** o monitor verá não apenas a lista de alunos, mas indicadores agregados sobre **engajamento por trilha** dentro de sua especialidade;
- **Sinalização de alerta:** servidores que não acessam a plataforma há determinado intervalo, ou cuja taxa de conclusão está significativamente abaixo da média, poderão ser sinalizados ao monitor, que poderá tomar iniciativa proativa de contato;
- **Comentários internos do monitor:** o monitor poderá manter **anotações privadas** (não visíveis ao aluno) sobre o acompanhamento, alimentando relatórios periódicos para a área de capacitação;
- **Articulação com lideranças:** dependendo do desenho institucional, o monitor poderá ser responsável por reportar **panoramas consolidados** à liderança da unidade, integrando capacitação à gestão.

10.5.13 Painel Administrativo

Funcionalidade-base: o painel administrativo da plataforma atual contempla: gerenciamento de disciplinas (`AdminDisciplines`, `AdminDisciplineEdit`), gerenciamento de monitores (`AdminMonitors`), relatórios (`AdminReports`), gerenciamento de usuários (`AdminUsers`) — incluindo o controle de **nível de acesso** introduzido na Entrega 2 deste período.

Especialização para o público interno: o painel administrativo do Capacita Portos Interno absorverá **todas essas capacidades** e adicionará atribuições inerentes ao caráter institucional:

- **Vínculo de servidor a unidade funcional:** o cadastro do servidor incluirá campos como **matrícula funcional, cargo, unidade de lotação, gerência e área de atuação**, alimentando filtros e relatórios;
- **Programa institucional de capacitação:** o admin poderá agrupar disciplinas em **programas de capacitação**, com objetivos formais, público-alvo declarado e indicadores de conclusão consolidados;
- **Designação de trilhas obrigatórias:** o admin poderá marcar determinadas disciplinas ou trilhas como **obrigatórias** para certos cargos ou áreas, e a interface do servidor passará a sinalizar pendências obrigatórias com clareza;
- **Agenda de capacitação:** disciplinas poderão ter **janelas de disponibilidade** (data de abertura e data de encerramento), em vez de estarem permanentemente disponíveis. Isso permite organizar **turmas** com início e fim definidos;
- **Auditoria administrativa:** todas as operações sensíveis do painel (alteração de papel, alteração de nível de acesso, criação de usuário, exclusão de conteúdo do fórum) serão registradas em um **log de auditoria** auditável, com identificação do operador, do alvo e do horário. Essa funcionalidade — não existente na plataforma atual — é **recomendação institucional padrão** para sistemas operados por administração pública.

10.5.14 Sistema de Níveis de Acesso

Funcionalidade-base: Sistema de níveis de acesso (Básico / Intermediário / Avançado) introduzido pela Entrega 2 deste período (capítulo 4), com gating de conteúdo por nível, persistência em coluna `access_level` de `user_roles`, RPC `set_user_access_level` para alteração administrativa e propagação por `AuthContext`.

Especialização para o público interno: o sistema de níveis de acesso será **integralmente aproveitado** e, possivelmente, **reconfigurado** para refletir a hierarquia funcional do quadro interno. Algumas possibilidades em discussão:

- **Manutenção dos três níveis nominais**, redefinindo o significado: Básico = todo o quadro; Intermediário = quadro técnico-finalístico; Avançado = quadro gerencial e especializado;
- **Substituição da nomenclatura** por termos mais coerentes com o domínio (por exemplo, Operacional / Técnico / Estratégico), com correspondente atualização dos rótulos em `ACCESS_LEVEL_LABELS`;
- **Expansão para múltiplos eixos:** considerar a introdução de **um terceiro eixo** de autorização, ortogonal a `role` e a `access_level`, indicando **área de competência** (Operações, Segurança, Ambiental, Administrativo etc.), com gating combinado.

Em todos os casos, três decisões arquiteturais herdadas devem ser observadas no projeto interno:

1. A proteção do conteúdo **deve** ser **server-side** desde o início (via RLS condicionada ao nível), não meramente client-side — débito identificado no capítulo 4 desta edição;
2. A RPC `get_my_access_level()` deve ser **efetivamente utilizada** pelo frontend, ou removida, evitando código morto;
3. A duplicação entre `accessLevels.js` (módulo de domínio) e listas locais em componentes administrativos deve ser **eliminada de origem**.

10.5.15 Exportação de Relatórios para Excel

Funcionalidade-base: função `exportStudentsToExcel` introduzida na Entrega 3 deste período (capítulo 5), gerando arquivo `.xlsx` com duas abas — Resumo e Detalhado — a partir dos dados administrativos. Dependência adicionada: `xlsx@^0.18.5`.

Especialização para o público interno: a exportação de dados terá importância **crítica** no contexto interno, dado o regime de **prestação de contas** e **transparência** da administração pública. Adaptações:

- **Múltiplos perfis de relatório:** além do relatório de alunos, o painel administrativo do Capacita Portos Interno poderá oferecer relatórios de **conclusão por área, trilhas obrigatórias por servidor, horas estimadas de capacitação realizadas, conformidade certificatória, acessos e auditoria**, entre outros;
- **Múltiplos formatos:** além de `.xlsx`, recomenda-se prever exportação em `.csv` (para integração com sistemas legados) e `.pdf` (para protocolização e arquivamento);
- **Carregamento sob demanda:** a recomendação registrada no capítulo 5 quanto ao *lazy loading* da rota administrativa e da biblioteca `xlsx` deve ser **incorporada de origem** no projeto interno, evitando que o bundle inicial do servidor comum carregue uma biblioteca que ele jamais usará;
- **Marca-d'água de identificação do operador:** ao gerar um relatório com PII de servidores, o arquivo poderá registrar, em **célula de cabeçalho** ou em metadado, **o nome do administrador que o gerou e o horário**, contribuindo para a rastreabilidade.

10.5.16 Moderação Administrativa do Fórum

Funcionalidade-base: políticas RLS introduzidas pela Entrega 4 deste período (capítulo 6), permitindo que o administrador exclua qualquer post ou resposta do fórum. O frontend já contemplava os botões correspondentes.

Especialização para o público interno: conforme já adiantado em 10.5.9, a moderação no fórum interno terá natureza institucional reforçada. Recomenda-se que **toda exclusão seja precedida de soft delete** e registrada em log de auditoria, e que monitores possam exercer moderação delegada.

10.5.17 Recuperação e Redefinição de Senha

Funcionalidade-base: fluxo de recuperação por e-mail (`forgot-password`) e tela de redefinição (`reset-password`), com redirecionamento configurável via `VITE_PASSWORD_RESET_REDIRECT_URL` . Componente `RecoveryRedirectHandler` interpreta o hash de URL no boot da aplicação.

Especialização para o público interno: o fluxo será mantido, com observações:

- **Domínio de e-mail validado:** apenas e-mails do domínio institucional poderão iniciar recuperação; tentativas com outros domínios serão silenciosamente ignoradas (sem revelar inexistência da conta — boa prática de privacidade);
- **Mensagem do e-mail personalizada** com identidade visual do Capacita Portos Interno;
- **Política de revogação rápida** em caso de incidente: o admin poderá, via RPC, **forçar reset de senha em massa** (mecanismo análogo ao já operacionalizado pela Entrega 1) caso seja necessário responder a um incidente de segurança.

10.5.18 Enforcement de Troca de Senha no Primeiro Acesso

Funcionalidade-base: flag `must_reset_password` em `raw_user_meta_data` do usuário, interpretada por `shouldResetPassword()` e propagada pelo `AuthContext` como `mustResetPassword` . O `ProtectedRoute` redireciona compulsoriamente para `/redefinir-senha` enquanto a flag estiver ativa. A Entrega 1 deste período operacionalizou o mecanismo para um conjunto específico de novos servidores.

Especialização para o público interno: o enforcement será **regra geral** para todo novo cadastro no Capacita Portos Interno, dado que as contas serão criadas pela administração com senha provisória.

10.5.19 Notificações e Badge de Dúvidas

Funcionalidade-base: o `Layout` mantém um badge numérico de notificação de dúvidas, atualizado por **polling de 30 segundos**.

Especialização para o público interno: conforme já recomendado, o projeto interno deve **substituir o polling por subscription em tempo real** via Supabase Realtime, ganho qualitativo significativo de responsividade e redução de tráfego desnecessário.

10.5.20 Layout Responsivo, Mobile e Sidebar

Funcionalidade-base: layout responsivo com **sidebar colapsável**, **menu mobile** e **overlay** (commits históricos `0613358` , `5dbf6e9`), oferecendo experiência consistente em desktop, tablet e celular.

Especialização para o público interno: a base responsiva será mantida e ampliada, considerando que servidores em áreas operacionais frequentemente acessam sistemas em **dispositivos móveis ou tablets corporativos** em campo. Atenção especial:

- **Modo de baixa largura de banda:** considerar uma variante visual otimizada para conexões corporativas restritas ou para uso em campo;
- **Compatibilidade com dispositivos corporativos:** garantir que a aplicação rode bem em navegadores **homologados pela TI institucional**, eventualmente com versões mais antigas que o estado-da-arte do mercado.

10.5.21 Dashboard do Aluno

Funcionalidade-base: o `Dashboard` apresenta ao aluno um panorama do seu progresso — total de disciplinas, disciplinas concluídas, em andamento, badges recentes — alimentado por consultas paralelas via `Promise.all()`.

Especialização para o público interno: o dashboard será personalizado para o servidor, exibindo:

- **Pendências obrigatórias** com destaque (trilhas que ainda precisam ser concluídas no prazo institucional);
- **Painel "Minhas Capacitações"** consolidando certificados emitidos;
- **Sugestões de próxima disciplina** com base na trilha de carreira ou área de atuação.

10.5.22 Sinalização de Disciplinas Concluídas

Funcionalidade-base: ajuste histórico (commit `5136cc2`) removeu o tachado (`line-through`) sobre títulos de aulas concluídas. O sistema usa marcação visual sóbria — ícone de check, cor distinta — para indicar conclusão.

Especialização para o público interno: a linguagem visual permanecerá sóbria, em coerência com a identidade institucional, evitando elementos lúdicos excessivos. A graduação cromática poderá ser ajustada à paleta oficial da autoridade portuária.

10.5.23 Política de Acesso e Segurança da Sessão

Funcionalidade-base: sessão gerenciada por Supabase Auth com JWT, listener `onAuthStateChanged` para mudanças, logout via `signOut`.

Especialização para o público interno:

- **Tempo de expiração de sessão** mais curto que o padrão, em conformidade com política de segurança da informação;
- **Logout automático por inatividade** após intervalo configurável;
- **Bloqueio em caso de IP fora da rede corporativa**, sempre que essa restrição for desejável (com exceções para acesso remoto autorizado).

10.6 Conteúdo Pedagógico Especializado — Eixos Temáticos

Embora o **catálogo de conteúdo** seja atribuição da área de capacitação da autoridade portuária e não da equipe técnica que desenvolve a plataforma, é útil registrar, ainda que de forma indicativa, os **eixos temáticos** que orientarão a curadoria inicial. Os eixos abaixo são apresentados como referência funcional para dimensionar o sistema:

1. **Eixo Institucional:** missão, visão, valores, estrutura organizacional, regimento interno, planejamento estratégico, transparência ativa, integridade e compliance, código de ética e conduta;
2. **Eixo Regulatório:** Lei dos Portos (Lei nº 12.815/2013) e regulamentação subsequente, atos da ANTAQ, atos da SEP/MPor, Normas da Autoridade Marítima (NORMAM), atos da Marinha do Brasil aplicáveis;
3. **Eixo Operacional:** operações de movimentação de carga, sinalização náutica, manobras de atracação, dragagem, manuseio de cargas perigosas (Código IMDG), interface com o agente marítimo, controle de janelas operacionais;
4. **Eixo de Segurança Portuária:** Código ISPS, certificação de instalações portuárias, gerenciamento de risco de segurança, controle de acesso, simulados, gestão de incidentes, interface com órgãos de segurança pública;
5. **Eixo de Segurança e Saúde Ocupacional:** NRs aplicáveis, especialmente NR-29 (Segurança e Saúde no Trabalho Portuário), uso de EPI, riscos específicos da atividade portuária, prevenção de acidentes, ergonomia;
6. **Eixo Ambiental:** licenciamento, gestão de resíduos, controle de efluentes e emissões, monitoramento de fauna e flora, gestão de áreas degradadas, plano de emergência ambiental, conformidade com a Política Nacional de Resíduos Sólidos;
7. **Eixo Administrativo:** Lei nº 14.133/2021 (Nova Lei de Licitações), gestão de contratos, gestão patrimonial, processo administrativo, prestação de contas, controle interno, gestão de pessoas e desenvolvimento de carreira;
8. **Eixo de Tecnologia da Informação:** Política de Segurança da Informação, LGPD aplicada à atividade portuária, governança de dados, controles de acesso, gestão de incidentes cibernéticos, política de uso aceitável;
9. **Eixo de Engenharia e Infraestrutura:** manutenção de infraestrutura de cais, dragagem, sinalização náutica, instalações elétricas e mecânicas portuárias, projetos de expansão e modernização, fiscalização técnica;
10. **Eixo Estatístico e de Inteligência Portuária:** indicadores de movimentação, análise de eficiência operacional, *benchmarking* portuário, métricas regulatórias da ANTAQ, transparência e *open data*.

Cada eixo poderá comportar **múltiplas disciplinas**, cada uma com **múltiplas aulas**, **quizzes individuais por aula** e **quiz final certificatório**, configurando trilhas que podem ter desde algumas horas até dezenas de horas de carga horária estimada.

10.7 Integração com Sistemas Internos — Possibilidades Futuras

O Capacita Portos Interno, embora se inicie como sistema **autônomo**, será projetado de modo a permitir, em fases futuras, **integrações com sistemas internos** da autoridade portuária. Possibilidades de integração consideradas (para roadmap):

- **Diretório institucional / SSO:** já mencionado em 10.5.1; centraliza ciclo de vida da identidade;
- **Sistema de gestão de pessoas:** envio automatizado de **registros de conclusão** para a ficha funcional do servidor;
- **Sistema de gestão eletrônica de documentos:** referência cruzada entre materiais publicados na plataforma e documentos protocolados;
- **Sistema de gestão por competências:** alimentação automática do **mapa de competências** do servidor a partir das disciplinas concluídas;
- **Datawarehouse / Business Intelligence corporativo:** exportação periódica de indicadores agregados para consumo em ferramentas de BI institucionais.

Todas essas integrações deverão ser desenhadas com **contratos versionados**, segregação clara de responsabilidades e auditoria, e jamais comprometendo a independência operacional da plataforma de e-learning.

10.8 Segurança Reforçada — Quadro Mínimo

Sintetizando as recomendações dispersas ao longo deste capítulo, o quadro mínimo de segurança para o Capacita Portos Interno compreende:

Domínio	Controle previsto
Identidade	Domínio institucional validado; SSO futuro; expiração de sessão curta
Autorização	Roles + access levels; RLS server-side em todas as tabelas; defesa em profundidade
Senhas	Política institucional; troca obrigatória no primeiro acesso; reset assistido
Dados pessoais	Tratamento como dado funcional; segregação física; políticas de retenção
Conteúdo sensível	Hospedagem controlada; URLs assinadas; marca-d'água em PDFs
API de IA	Backend proxy mandatório; sem chave embarcada no frontend
Auditoria	Log estruturado de operações administrativas e moderação
Tempo real	Supabase Realtime substituindo polling; menor latência
Backups	Snapshots regulares; testes de restauração periódicos
Incidentes	Plano de resposta documentado; procedimento de revogação rápida

10.9 Conformidade — LGPD e Princípios da Administração Pública

O tratamento de dados pessoais no Capacita Portos Interno será orientado pela **Lei Geral de Proteção de Dados (LGPD)** e pelos **princípios da administração pública** (legalidade, impessoalidade, moralidade, publicidade, eficiência e, conforme jurisprudência, proporcionalidade, razoabilidade e transparência). Diretrizes fundamentais:

- **Base legal de tratamento:** o tratamento de dados dos servidores ocorrerá com base nas hipóteses pertinentes da LGPD (execução de política pública, cumprimento de obrigação legal, legítimo interesse), devidamente documentada;
- **Minimização:** coleta apenas dos dados necessários à finalidade declarada;
- **Transparência:** publicação de **aviso de privacidade** acessível e claro, descrevendo dados coletados, finalidades, retenção e direitos do titular;
- **Direitos do titular:** procedimentos para atendimento de solicitações de acesso, correção, eliminação, portabilidade e revogação;
- **Encarregado (DPO):** o Capacita Portos Interno se integrará ao **encarregado da autoridade portuária**, com canal claro para titulares;
- **Retenção:** definição de **prazos máximos** de retenção de dados, com expurgo automatizado ao final do período;
- **Não reuso indevido:** o histórico de capacitação **não poderá** ser utilizado para fins não declarados, especialmente não poderá ser empregado de forma punitiva sem que isso esteja previamente estabelecido em política institucional;
- **Auditoria por órgão de controle:** os dados e logs estarão disponíveis a auditorias por órgãos de controle interno e externo, sempre que requisitados nos termos da legislação.

10.10 Cronograma e Marcos — Visão Indicativa

Apresenta-se, a título indicativo (e sujeito a confirmação pela coordenação do projeto), uma visão preliminar de fases, sem datas absolutas:

Fase	Marco
Fase 0	Definição de requisitos institucionais e identidade visual do Capacita Portos Interno
Fase 1	Bootstrap do repositório com TypeScript, suíte de testes e <i>backend proxy</i> de IA desde o primeiro commit
Fase 2	Replicação das funcionalidades-base (autenticação, disciplinas, aulas, quizzes, badges, fórum, dúvidas, IA)
Fase 3	Replicação das funcionalidades administrativas (admin, monitor, relatórios, níveis de acesso, exportação)
Fase 4	Funcionalidades específicas do contexto interno (certificados, trilhas obrigatórias, auditoria, log)
Fase 5	Curadoria de conteúdo, importação dos primeiros materiais, criação das primeiras disciplinas
Fase 6	Piloto fechado com um grupo restrito de servidores
Fase 7	Liberação institucional ampla e abertura à totalidade do quadro
Fase 8	Integrações com sistemas internos (SSO, gestão de pessoas etc.)

10.11 Riscos e Mitigações Específicas

Risco	Probabilidade	Impacto	Mitigação
Reprodução das dívidas técnicas da plataforma atual	Média	Alto	Adoção, desde o primeiro commit, de TypeScript, suíte de testes, backend proxy de IA e Realtime
Sobreposição confusa com a plataforma pública	Baixa	Médio	Identidade visual, terminologia e governança próprias; comunicação institucional clara
Resistência cultural ao registro de desempenho funcional	Média	Médio	Transparência sobre uso dos dados; ranking opt-in; vínculo a valorização, não a punição
Sobrecarga da curadoria pedagógica	Alta	Médio	Roteiro de produção; reuso de materiais existentes; envolvimento de servidores como autores
Custo de IA escalando com a base de usuários	Média	Médio	Quotas, cache, prompts otimizados, possibilidade de troca de provedor
Não conformidade com LGPD	Baixa	Alto	Aviso de privacidade; mínimos necessários; integração com encarregado
Falha de integração com SSO / diretório	Baixa	Médio	Desenho com fallback; integração apenas em fase consolidada

10.12 Sinergia com o Projeto Atual — Estratégia de Código Compartilhado

Considerando que as duas plataformas compartilham **arquitetura e a maior parte da lógica de negócio**, é altamente recomendável adotar **uma estratégia explícita de compartilhamento de código** entre os dois projetos, com três opções possíveis:

1. **Monorepo:** ambos os projetos coexistem no mesmo repositório, com pacotes internos compartilhados (por exemplo, um pacote `@capacita/core` contendo `badges`, `accessLevels`, `supabase`, etc.). Vantagem: refatoração simultânea. Desvantagem: governança e CI mais complexas;
2. **Repositórios separados com biblioteca privada compartilhada:** cada plataforma fica em seu repositório, e ambos consomem uma biblioteca interna (publicada em registry privado). Vantagem: independência de cadência. Desvantagem: overhead de publicação;
3. **Repositórios separados sem compartilhamento:** cada plataforma evolui de forma independente. Vantagem: simplicidade. Desvantagem: duplicação massiva e divergência ao longo do tempo.

A **opção recomendada** é a (1) ou a (2), priorizando-se a (2) caso já exista cultura de bibliotecas internas na organização, e a (1) caso contrário. A opção (3) **não é recomendada** — perpetuaria divergência e desperdício.

10.13 Conclusão do Capítulo

O **Capacita Portos Interno** representa uma evolução natural e estrategicamente relevante do ecossistema Capacita Portos, ampliando o alcance da plataforma para o **quadro funcional da autoridade portuária** e materializando, em formato digital, gamificado e mensurável, o esforço institucional de **capacitação contínua, qualificação técnica e desenvolvimento dos servidores**.

A iniciativa parte de uma posição privilegiada: o **catálogo de funcionalidades já está maduro** na plataforma atual, **a arquitetura é comprovadamente extensível** (conforme demonstrado nos capítulos 1 a 9 desta edição), e a **equipe técnica acumulou aprendizados claros** sobre o que reproduzir e o que evitar. Em especial, o projeto interno tem a **oportunidade histórica** de nascer **já corrigido** dos seis pontos críticos herdados da plataforma atual — ausência de testes, exposição da chave de IA, complexidade ciclomática, tratamento inconsistente de erros, ausência de tipagem estática e ausência de sincronização em tempo real. Recomenda-se que essa oportunidade não seja perdida.

Como peça final deste capítulo, registra-se a **expectativa** de que a próxima edição deste relatório (terceira edição) possa documentar não apenas a evolução da plataforma atual, mas **também o nascimento e o ciclo de vida inicial** do Capacita Portos Interno, oferecendo, em um único documento técnico de referência, a fotografia consolidada de **ambas as plataformas** do ecossistema Capacita Portos.

11. INVENTÁRIO DE TECNOLOGIAS, PACOTES E BIBLIOTECAS

Este capítulo apresenta, de forma **sistemática, completa e detalhada**, o inventário de todas as tecnologias, linguagens, bibliotecas, pacotes, ferramentas e padrões empregados na construção, na operação e na manutenção da plataforma Capacita Portos. O objetivo é constituir uma **referência técnica de consulta direta**, útil tanto para a equipe atual quanto para futuros desenvolvedores, auditores e mantenedores. Cada item é apresentado com sua identificação precisa, versão em uso, papel arquitetural, justificativa de adoção e — quando pertinente — alternativas consideradas, riscos associados e recomendações.

11.1 Visão Geral do Stack Tecnológico

A plataforma Capacita Portos é construída sobre um stack moderno, orientado a Single Page Application (SPA), com clara separação entre frontend e backend-as-a-service (BaaS). Em uma única frase descritiva, trata-se de uma **aplicação React com Vite, hospedada em provedor estático, integrada a Supabase como BaaS (PostgreSQL gerenciado, Auth, Storage e RLS) e a Google Generative AI (Gemini) como provedor de LLM para o assistente conversacional**.

O quadro a seguir oferece uma síntese de alto nível, agrupando as tecnologias por camada arquitetural:

Camada	Tecnologia(s) principal(is)	Papel
Linguagens	JavaScript (ES2020+), JSX, HTML5, CSS3, SQL (PostgreSQL), Python 3	Sintaxe e expressão do código
Framework UI	React 19	Composição declarativa de interfaces
Roteamento	React Router DOM 7	Navegação SPA com rotas aninhadas
Build / Dev Server	Vite 7	Bundler ESM nativo e servidor de desenvolvimento
Plugin de build	@vitejs/plugin-react	Suporte a JSX e Fast Refresh
Qualidade de código	ESLint 9 + plugins	Linting e enforcement de boas práticas
Tipagem (definições)	@types/react, @types/react-dom	Type hints em editor
BaaS	Supabase	Auth, banco, RLS, Storage
Cliente do BaaS	@supabase/supabase-js	Acesso programático ao Supabase
Banco de dados	PostgreSQL (gerenciado pelo Supabase)	Persistência relacional
LLM	Google Generative AI (Gemini)	Assistente conversacional
Cliente do LLM	@google/generative-ai	Acesso programático ao Gemini
Iconografia	React Icons	Conjunto unificado de ícones SVG
Manipulação de planilhas	xlsx (SheetJS)	Exportação de relatórios em <code>.xlsx</code>
Scripts auxiliares	Python 3	Geração programática de SQL
Versionamento	Git + GitHub	Controle de versão e colaboração
Runtime de desenvolvimento	Node.js	Execução do tooling (Vite, ESLint)

Os itens acima estão **efetivamente declarados** em `package.json`, em `vite.config.js`, em `eslint.config.js`, em `.env.example` e no diretório `supabase/`. As seções a seguir os documentam um a um.

11.2 Linguagens de Programação e Marcação

11.2.1 JavaScript (ECMAScript 2020+)

A linguagem-base do código de aplicação é o **JavaScript moderno**, no nível **ECMAScript 2020 e posteriores**, conforme configurado em `eslint.config.js` por meio de `ecmaVersion: 2020` (com `ecmaVersion: 'latest'` em `parserOptions`). Recursos amplamente utilizados ao longo da base de código incluem:

- **Módulos ES (ESM)** — `import / export`, com `"type": "module"` declarado em `package.json`;
- **Funções de seta (arrow functions) e closures**;
- **Desestruturação** de objetos e arrays;
- **Operador de espalhamento (spread) e rest parameters**;
- **Templates de string** (template literals) com interpolação;
- **Operadores `??` (nullish coalescing) e `?.` (optional chaining)**, este último particularmente útil em acessos a respostas do Supabase;
- **`async / await`** como forma idiomática de tratamento de promises;
- **`Promise.all()`** para paralelização de requisições — padrão extensivamente utilizado nas funções `fetchData()` de páginas como `Dashboard` e `DisciplineDetail`;
- **Classes** (uso esparsa) e **funções declarativas** como padrão dominante;
- **Sets e Maps** (`new Set()`, `new Map()`) — utilizados, por exemplo, em `computeDisciplineBadges` para conjuntos de IDs de aulas concluídas.

Justificativa de adoção: o JavaScript é a linguagem nativa do navegador e do ecossistema React, dispensa transpilação adicional além daquela já realizada pelo Vite, e oferece a curva de aprendizado mais acessível ao maior contingente de desenvolvedores web. A escolha por ES2020+ é coerente com os navegadores-alvo modernos suportados pela aplicação.

Ponto de atenção: a primeira edição deste relatório registrou, como ponto crítico nº 5, a **ausência de tipagem estática (TypeScript)**. O código permanece tipado apenas via JSDoc esparsa, sem verificação estática. A adoção de TypeScript continua sendo recomendação relevante, conforme já consolidado nas conclusões do capítulo 9 e ratificado no capítulo 10 como oportunidade arquitetural para o projeto Capacita Portos Interno.

11.2.2 JSX (JavaScript XML)

O **JSX** é a extensão de sintaxe utilizada para descrever a árvore de componentes React. Não é uma linguagem separada, mas uma extensão sintática transpilada para chamadas `React.createElement` (ou, em React 17+, para o *automatic runtime*). Todos os arquivos `.jsx` da aplicação utilizam JSX para construir a UI declarativamente. A configuração do parser ESLint inclui `ecmaFeatures: { jsx: true }`, habilitando o reconhecimento da sintaxe.

11.2.3 HTML5 e CSS3

A camada de apresentação utiliza **HTML5** (estrutura semântica) e **CSS3** (estilos), com as seguintes características:

- **HTML5 semântico** — uso de elementos `<header>`, `<nav>`, `<main>`, `<section>`, `<aside>`, `<footer>` quando apropriado;
- **CSS modular por componente** — cada componente importante possui seu próprio arquivo `.css` colocado ao lado do `.jsx` correspondente (por exemplo, `DisciplineDetail.jsx` e `DisciplineDetail.css`). Esse padrão organiza o estilo por responsabilidade e evita o acúmulo de uma única folha de estilo global gigantesca;
- **Flexbox e Grid** — utilizados de forma extensiva para layouts responsivos, incluindo o sistema de abas, painéis, cartões de disciplina e tabelas administrativas;
- **Variáveis CSS / cores institucionais** — a paleta dominante inclui o teal corporativo `#009b8f` (e suas variantes em estados de hover, como `#007a72`), em conjunto com tons neutros (`#f1f5f9`, `#e5e7eb`, `#6b7280`) e cores de alerta (`#94560`, `#fee`) — todas observáveis nos arquivos CSS analisados;
- **Animações sutis** — micro-interações como `transform: translateY(1px)` em estado `:active` de botões, transições de cor em hover, animações de conquistas (Badge Unlocked);
- **Media queries** — adaptação responsiva para tablet e celular, presente nos arquivos `Layout.css`, `DisciplineDetail.css` e demais.

11.2.4 SQL (PostgreSQL Dialect)

A linguagem **SQL**, no dialeto **PostgreSQL**, é utilizada de forma intensa no diretório `supabase/`, que contém **25 arquivos** entre `schema.sql`, migrações nomeadas (`migration_*.sql`) e scripts operacionais (`sql_*.sql`). Recursos do PostgreSQL extensivamente empregados:

- **Tipos enumerados (ENUM)** — exemplo notório: `access_level_enum` criado pela migração de níveis de acesso (Entrega 2);
- **jsonb e operadores associados** — utilizados em `raw_user_meta_data` da tabela `auth.users`, com operadores `->`, `->>` (acesso a campos), `||` (concatenação/mesclagem) e `COALESCE` para tratamento de nulos;
- **Row Level Security (RLS)** — políticas declarativas que controlam acesso linha a linha; presentes em todas as tabelas operacionais, incluindo as políticas de moderação introduzidas pela Entrega 4 deste período;
- **Funções plpgsql** — escritas com `LANGUAGE plpgsql` e, em casos de operação privilegiada, com modificador `SECURITY DEFINER` (com a devida verificação de papel via `is_admin()`);
- **CREATE OR REPLACE FUNCTION** — viabiliza versionamento iterativo de funções RPC sem perda de referências;

- `DO $$ ... $$` — blocos anônimos para lógica condicional (por exemplo, criação idempotente de tipos);
- `ON CONFLICT DO UPDATE / DO NOTHING` — implementação do padrão *upsert*, utilizada em diversas migrações e scripts;
- `RAISE EXCEPTION` — interrupção controlada para validações em funções RPC;
- **Cláusulas** `IF NOT EXISTS` — empregadas para tornar migrações idempotentes.

A presença de `schema.sql` na raiz do diretório `supabase/` indica que o esquema completo é mantido como artefato consultável, complementado pelas migrações que registram alterações incrementais.

11.2.5 Python 3

O **Python 3** aparece como **linguagem auxiliar de scripting** — não como linguagem da aplicação. O arquivo `generate_sql.py` (introduzido pela Entrega 1 deste período e detalhado no capítulo 3) é um gerador descartável que produz comandos SQL a partir de listas de e-mails. O Python é escolhido neste contexto pela sua **brevidade para tarefas pontuais de manipulação de texto** e pela ampla disponibilidade em ambientes administrativos. Sua utilização é isolada e não impõe dependência operacional sobre a aplicação principal.

11.2.6 Bash / Shell

Scripts pontuais e comandos de operação (deploy, instalação) utilizam **Bash/Shell**, em conformidade com o ambiente Linux dos servidores e do ambiente de desenvolvimento. O histórico de commits revela passagens pelo workflow de deploy via SSH (commits `762ac1e`, `86c7d51`, `a5743a2`, posteriormente revertido em `416289c`).

11.3 Framework Principal: React 19

11.3.1 React (`react`) — versão `^19.2.4`

O **React** é a biblioteca-base de construção de interfaces de usuário, na versão **19.2.4**. Esta é uma versão moderna e estável que incorpora avanços relevantes em relação às versões anteriores:

- **Automatic Batching** — múltiplas atualizações de estado disparadas no mesmo ciclo são agrupadas automaticamente, reduzindo renderizações desnecessárias;
- **Concurrent Features** — suporte a `useTransition`, `useDeferredValue` e `Suspense`, embora a aplicação ainda não os utilize de forma intensiva;
- **Improved Suspense Boundaries** — melhor tratamento de fronteiras de carregamento;
- **Server Components** — não utilizados nesta aplicação (que é cliente-only), mas disponíveis no roadmap caso a arquitetura evolua para um modelo full-stack.

Padrões e APIs efetivamente utilizados na base:

- `useState`, `useEffect`, `useContext`, `useRef`, `useCallback`, `useMemo` — hooks fundamentais;
- `createContext` e `Provider` — utilizados em `AuthContext`;
- `React.StrictMode` (em `main.jsx`) — habilita verificações adicionais em desenvolvimento;
- Renderização condicional via `&&` e ternários (`?:`);
- Listas via `map` com chaves estáveis (atributo `key`).

11.3.2 React DOM (`react-dom`) — versão `^19.2.4`

O **React DOM** é o pacote responsável pela renderização de componentes React no DOM real do navegador. A API utilizada é `createRoot` (do React 18+) em `src/main.jsx`, na forma:

```
import { createRoot } from 'react-dom/client'
createRoot(document.getElementById('root')).render(<App />)
```

A versão acompanha rigorosamente a versão de `react`, conforme requisito do framework.

11.3.3 React Router DOM (`react-router-dom`) — versão `^7.13.0`

O **React Router DOM v7** provê o sistema de roteamento da SPA. APIs e padrões utilizados:

- `BrowserRouter` — modo de roteamento que utiliza a API History do HTML5;
- `Routes` e `Route` — declaração das rotas;
- **Rotas aninhadas (nested routes)** — utilizadas para envolver páginas internas no `Layout` e em `ProtectedRoute`;
- `Navigate` — redirecionamento declarativo (usado nas rotas de proteção);
- `useParams` — recuperação de parâmetros de rota (por exemplo, `id` em `/disciplina/:id`);
- `useNavigate` — navegação programática;
- `useLocation` — acesso à localização atual (utilizado em handlers de redirecionamento).

A versão 7 é a versão atual com suporte a Future Flags, e mantém retrocompatibilidade com o estilo declarativo da versão 6. A escolha é coerente com o padrão de mercado para SPAs React.

11.3.4 React Icons (`react-icons`) — versão `^5.5.0`

O pacote **React Icons** agrupa, em um único pacote, **dezenas de coleções de ícones vetoriais** (Feather Icons, Font Awesome, Material Design, etc.), expostos como componentes React. Vantagens da escolha:

- **Tree-shaking nativo** — apenas os ícones efetivamente importados (`import { FiTrash2 } from 'react-icons/fi'`) entram no bundle final;

- **Consistência tipográfica** — todos os ícones seguem o mesmo padrão de uso, independentemente da família de origem;
- **Cobertura ampla** — a aplicação utiliza predominantemente a coleção **Feather** (`fi`), por sua estética minimalista compatível com o restante da identidade visual.

Ícones presentes na base de código (entre outros): `FiPlay`, `FiFileText`, `FiCheckCircle`, `FiLock`, `FiCheck`, `FiX`, `FiMessageCircle`, `FiDownload`, `FiEdit3`, `FiBookOpen`, `FiArrowLeft`, `FiSave`, `FiPlus`, `FiTrash2`, `FiEdit2`, `FiUpload`, `FiFile`, `FiUsers`, `FiBook`, `FiAward`, `FiTrendingUp`, `FiChevronDown`, `FiChevronUp`, `FiBarChart2`, `FiClock`, `FiPercent`.

11.4 Ferramentas de Build, Bundling e Desenvolvimento

11.4.1 Vite (`vite`) — versão `^7.3.1`

O **Vite** é o bundler e servidor de desenvolvimento adotado, na versão **7.3.1**. Suas características relevantes para o projeto:

- **Servidor de desenvolvimento baseado em ESM nativo** — durante o desenvolvimento, módulos são servidos individualmente ao navegador via `import` ESM, sem a necessidade de bundling prévio. Isso resulta em **tempos de cold start abaixo de 1 segundo** mesmo em projetos médios;
- **Hot Module Replacement (HMR) sub-100ms** — alterações em arquivos `.jsx` ou `.css` propagam-se ao navegador em frações de segundo, preservando o estado da aplicação;
- **Build de produção via Rollup** — para produção, o Vite delega ao Rollup, que produz bundles otimizados, com tree-shaking, minificação e code splitting automático;
- **Configuração mínima** — `vite.config.js` da aplicação tem apenas 14 linhas, declarando o plugin React e configurando host/porta do servidor;
- **Suporte nativo a TypeScript, JSX, CSS Modules, importação de assets** — sem configuração adicional necessária.

A configuração específica do projeto (`vite.config.js`):

```
export default defineConfig({
  plugins: [react()],
  server: {
    host: '0.0.0.0',
    port: 8571,
    strictPort: true,
  },
  preview: {
    host: '0.0.0.0',
    port: 8571,
    strictPort: true,
  },
})
```

A porta fixa `8571` (com `strictPort: true`, que falha em vez de mudar de porta) facilita a configuração de reverse proxies e de regras de firewall em ambientes de homologação e produção. O host `0.0.0.0` permite conexões de fora da máquina local, útil para testes em rede.

Scripts disponíveis (em `package.json`):

- `npm run dev` — inicia o servidor de desenvolvimento;
- `npm run build` — gera o bundle de produção em `dist/`;
- `npm run preview` — serve o bundle de produção localmente para testes;
- `npm run lint` — executa o ESLint.

11.4.2 Plugin Vite para React (`@vitejs/plugin-react`) — versão `^5.1.4`

Plugin oficial que adiciona ao Vite o suporte a **JSX**, ao **Fast Refresh** (preservação de estado do componente durante o HMR) e à integração com Babel quando necessário. Sem este plugin, o Vite não reconheceria automaticamente `.jsx`.

11.5 Qualidade de Código e Linting

11.5.1 ESLint (`eslint`) — versão `^9.39.2`

O **ESLint** é o linter empregado, na **versão 9** (flat config). A configuração unificada em `eslint.config.js` define:

- **Ignore global** de `dist/` (artefatos de build);
- **Escopo** — todos os arquivos `.js` e `.jsx` do projeto;
- **Extends** — recomendações da `@eslint/js`, `react-hooks` e `react-refresh`;
- **Language Options** — ECMAScript 2020, globals do navegador, parser com suporte a JSX e `sourceType: 'module'`;

- **Rules customizadas** — `no-unused-vars` configurada para **ignorar variáveis iniciadas em maiúscula ou underscore** (`varsIgnorePattern: '^[A-Z_]'`), padrão coerente com o ecossistema React (onde componentes começam com maiúscula).

A adoção da **flat config** (formato moderno do ESLint 9) substitui o antigo `.eslintrc`, simplificando a estrutura e tornando a configuração programaticamente clara.

11.5.2 ESLint JS (`@eslint/js`) — versão `^9.39.2`

Pacote que fornece o conjunto de regras recomendadas (`js.configs.recommended`) para JavaScript moderno. É a base sobre a qual outros plugins são empilhados.

11.5.3 ESLint Plugin React Hooks (`eslint-plugin-react-hooks`) — versão `^7.0.1`

Plugin que **detecta violações das Rules of Hooks** do React — como chamar hooks dentro de condicionais, em laços, ou fora do top-level de funções de componente. É uma camada de proteção crucial: violações dessas regras geram bugs sutis e difíceis de depurar, e o plugin os captura ainda em tempo de desenvolvimento.

11.5.4 ESLint Plugin React Refresh (`eslint-plugin-react-refresh`) — versão

`^0.4.26`

Plugin específico para projetos Vite + React. Garante que os componentes sejam **compatíveis com Fast Refresh**, alertando sobre padrões que romperiam a capacidade do HMR de preservar o estado durante recargas.

11.5.5 Globals (`globals`) — versão `^16.5.0`

Pacote utilitário que provê listas de **identificadores globais** conhecidos por ambiente (`browser`, `node`, etc.), usadas pelo ESLint para reconhecer variáveis como `window`, `document`, `console` sem marcá-las como indefinidas. A configuração do projeto utiliza `globals.browser`.

11.6 Definições de Tipo (TypeScript Definitions)

Apesar de a aplicação não ser escrita em TypeScript, o `package.json` declara pacotes `@types/` *. Essas definições não impõem tipagem estática à base de código, mas oferecem **IntelliSense / autocomplete tipado** em editores compatíveis (VS Code, JetBrains), além de informarem ao ESLint sobre as APIs disponíveis.

11.6.1 `@types/react` — versão `^19.2.14`

Definições de tipo para a API do React, alinhadas à versão 19.

11.6.2 @types/react-dom — versão ^19.2.3

Definições de tipo para o React DOM (incluindo `createRoot`).

Observação: a presença dessas definições, mesmo sem TypeScript, é um sinal de **intenção arquitetural de habilitar tipagem futura** — uma transição para TypeScript, conforme recomendado no capítulo 9, seria facilitada pela presença prévia desses tipos.

11.7 Backend-as-a-Service: Supabase

11.7.1 Visão Geral do Supabase

O **Supabase** é o **backend-as-a-service** sobre o qual toda a camada de persistência, autenticação, autorização e armazenamento de arquivos da plataforma se assenta. Trata-se de uma plataforma open source que entrega, como produto unificado, os seguintes serviços:

- **PostgreSQL gerenciado** — instância de banco relacional moderno (versão 15+), com todos os recursos do PostgreSQL disponíveis ao desenvolvedor;
- **Autenticação (Supabase Auth)** — gerenciamento de usuários, JWT, recuperação de senha, providers sociais (não utilizados pela aplicação atual);
- **Row Level Security (RLS)** — uso do mecanismo nativo do PostgreSQL para autorização declarativa por linha;
- **Storage** — armazenamento de arquivos (utilizado para materiais complementares de disciplinas);
- **Edge Functions** — funções serverless (Deno) — **não utilizadas atualmente**, mas previstas como solução para o débito da chave de IA (ver capítulo 9 e 10);
- **Realtime** — assinaturas em tempo real via WebSocket — **não utilizadas atualmente** (ponto crítico nº 6 da primeira edição).

A adoção do Supabase é a **decisão arquitetural de maior impacto** do projeto: ela substitui simultaneamente um backend customizado, uma infraestrutura de autenticação e um mecanismo de autorização, reduzindo dramaticamente a superfície de código operacional a ser mantida pela equipe.

11.7.2 Cliente Supabase JS (@supabase/supabase-js) — versão ^2.95.3

O **cliente oficial** do Supabase para JavaScript/TypeScript, na versão **2.95.3**. A aplicação utiliza este cliente em **instância singleton** (padrão identificado na primeira edição, seção 1.1.4):

```
import { createClient } from '@supabase/supabase-js'
export const supabase = createClient(
  import.meta.env.VITE_SUPABASE_URL,
  import.meta.env.VITE_SUPABASE_ANON_KEY
)
```

APIs do cliente utilizadas extensivamente:

- **Auth** — `supabase.auth.signInWithPassword`, `signUp`, `signOut`, `resetPasswordForEmail`, `updateUser`, `getSession`, `onAuthStateChange`;
- **Database (PostgREST)** — `supabase.from('tabela').select().insert().update().upsert().delete()`, com `.eq()`, `.in()`, `.order()`, `.single()`, `.limit()` etc.;
- **RPC** — `supabase.rpc('nome_da_funcao', { parametros })` — utilizada para chamadas a funções `plpgsql` do banco, como `set_user_access_level` e `get_platform_users`;
- **Storage** — `supabase.storage.from('bucket').upload().download().getPublicUrl()`, utilizada na funcionalidade de upload de materiais.

11.7.3 PostgreSQL como Banco de Dados

O banco de dados subjacente é o **PostgreSQL**, gerenciado pelo Supabase. Tabelas operacionais identificadas (a partir das migrações em `supabase/`):

Tabela	Função
<code>auth.users</code>	Tabela do Supabase Auth com credenciais e metadados de usuário
<code>user_roles</code>	Vínculo de usuário a papel (<code>role</code>) e nível de acesso (<code>access_level</code>)
<code>disciplines</code>	Disciplinas da plataforma
<code>lessons</code>	Aulas vinculadas a disciplinas
<code>materials</code>	Materiais complementares por disciplina
<code>lesson_progress</code>	Registro de aulas concluídas por usuário
<code>user_progress</code>	Registro de disciplinas concluídas por usuário
<code>quiz_results</code>	Resultado do quiz final por (usuário, disciplina)
<code>lesson_quiz_results</code>	Resultado do quiz de aula por (usuário, aula)
<code>forum_posts</code>	Posts do fórum
<code>forum_replies</code>	Respostas a posts do fórum
<code>doubts</code>	Dúvidas enviadas pelos alunos
<code>doubt_responses</code>	Respostas dos monitores às dúvidas

Funções RPC identificadas:

- `is_admin()` — verifica se o usuário corrente é administrador;
- `get_platform_users()` — retorna a lista de usuários da plataforma (com `role` e `access_level`), restrita a administradores;
- `get_my_access_level()` — retorna o nível de acesso do usuário corrente (introduzida na Entrega 2 mas não utilizada pelo frontend);
- `set_user_access_level(p_user_id, p_access_level)` — altera o nível de acesso de um usuário (restrita a administradores).

Recursos avançados em uso: `ENUM`, `RLS`, `SECURITY DEFINER`, `plpgsql`, `jsonb`, `ON CONFLICT`, `RAISE EXCEPTION`, índices implícitos via chaves primárias e estrangeiras, e `transactions` (`BEGIN` / `COMMIT`).

11.7.4 Variáveis de Ambiente Relativas ao Supabase

Definidas em `.env.example` (e configuradas no ambiente real do desenvolvedor/produção):

- `VITE_SUPABASE_URL` — URL do projeto Supabase (formato `https://<id>.supabase.co`);
- `VITE_SUPABASE_ANON_KEY` — chave pública anônima do projeto, embarcada no bundle (uso esperado da chave anon, que confia nas políticas RLS para autorização).

11.8 Inteligência Artificial

11.8.1 Google Generative AI (@google/generative-ai) — versão ^0.24.1

Cliente JavaScript oficial do **Google Generative AI**, utilizado pela aplicação para integrar o **Google Gemini** ao assistente conversacional (componente `AIChat`). APIs principais utilizadas (encapsuladas em `src/lib/gemini.js`):

- `GoogleGenerativeAI(apiKey)` — instanciação do cliente;
- `getGenerativeModel({ model: '...' })` — seleção do modelo (tipicamente `gemini-pro` ou `gemini-1.5-flash`);
- `startChat({ history, systemInstruction })` — início de uma sessão de chat com contexto persistente;
- `sendMessage(message)` — envio de mensagem e recebimento de resposta.

A integração utiliza **system prompt contextualizado por disciplina**, restringindo o escopo da conversa ao conteúdo da disciplina atual — uma forma de **mitigação de prompt injection** já analisada na primeira edição.

11.8.2 Variável de Ambiente do Gemini

Definida em `.env.example`:

- `VITE_GEMINI_API_KEY` — chave de API do Google Generative AI.

Alerta crítico (herdado da primeira edição e ainda não resolvido): por se tratar de uma variável prefixada com `VITE_`, esta chave é **embarcada no bundle de frontend** e fica **exposta ao público** após o build. Esta é a vulnerabilidade nº 2 da primeira edição, **não endereçada** durante o período coberto por esta segunda edição, e classificada como **prioridade P0** nas recomendações do capítulo 9.

11.9 Manipulação de Dados e Exportação

11.9.1 xlsx (SheetJS) — versão ^0.18.5

Biblioteca para **leitura e escrita de planilhas em formato Excel** (`.xlsx`, `.xls`, `.csv` e outros). Foi introduzida no projeto pela Entrega 3 deste período (capítulo 5), exclusivamente para a funcionalidade de **exportação de relatórios administrativos**. APIs utilizadas:

- `XLSX.utils.book_new()` — cria uma nova pasta de trabalho (workbook) vazia;
- `XLSX.utils.json_to_sheet(array)` — converte um array de objetos JavaScript em uma planilha;
- `XLSX.utils.book_append_sheet(wb, ws, nome)` — adiciona uma planilha à pasta de trabalho;

- `ws['!cols']` — atribuição direta para definir larguras de coluna;
- `XLSX.writeFile(wb, nome)` — escreve o arquivo e dispara o download no navegador.

Considerações: a versão `0.18.5` é uma versão pública madura. A aplicação usa a biblioteca **exclusivamente para escrita** (não lê planilhas de entrada do usuário), o que reduz a superfície de risco. Recomenda-se monitoramento contínuo de avisos de segurança e avaliação de atualização para a versão mais recente do mantenedor, conforme detalhado no capítulo 5.

11.10 Variáveis de Ambiente — Inventário Completo

O arquivo `.env.example` (template versionado, com valores fictícios) declara o conjunto completo de variáveis de ambiente esperadas pela aplicação:

Variável	Tipo	Embarcada no bundle?	Função
<code>VITE_SUPABASE_URL</code>	URL	Sim (necessário)	Endpoint do projeto Supabase
<code>VITE_SUPABASE_ANON_KEY</code>	Chave pública	Sim (esperado para anon key)	Acesso ao Supabase mediado por RLS
<code>VITE_GEMINI_API_KEY</code>	Chave de API	Sim — vulnerabilidade	Acesso ao Google Gemini
<code>VITE_PASSWORD_RESET_REDIRECT_URL</code>	URL	Sim	URL para redirecionamento pós-reset de senha (ex.: <code>https://capacitaportos.com.br/redefinir-senha</code>)

O prefixo `VITE_` é **convenção do Vite** para variáveis que devem ser expostas ao código de cliente. Quaisquer variáveis sensíveis deveriam, idealmente, **não** ter este prefixo — caso da `VITE_GEMINI_API_KEY`, cuja correção exige migração para um backend proxy (Supabase Edge Function), conforme já recomendado.

11.11 Sistema de Versionamento e Colaboração

11.11.1 Git

O **Git** é o sistema de controle de versão adotado. O repositório acompanha histórico completo desde os primeiros commits do projeto, com cinco entregas relevantes no período analisado por esta edição (capítulos 3 a 7). Práticas observadas:

- **Mensagens de commit em padrão Conventional Commits** — prefixos como `feat:`, `fix:`, `refactor:`, `docs:`, `test:`;
- **Merges de branches** — histórico mostra merges de branches como `fork/imgbot`, `fork/copilot/...` e merges de pull requests;
- **Cadência regular** — média de uma entrega significativa a cada duas semanas no período recente.

11.11.2 GitHub

O repositório é hospedado no **GitHub**, conforme indicado pelos merges de pull requests visíveis no histórico (`Merge pull request #1`, `Merge pull request #2`). O GitHub é também a plataforma utilizada para revisão de código e, no histórico do projeto, para automação de workflows.

11.11.3 GitHub Actions / Workflows

O histórico de commits revela passagens por **GitHub Actions** para automação de deploy:

- Commits `762ac1e`, `86c7d51` e `a5743a2` introduziram um workflow de deploy via SSH;
- O commit `416289c` posteriormente removeu esse workflow ("Refactor: remover workflow de deploy via SSH"), indicando uma reorientação da estratégia de deploy.

A estratégia de deploy atual não está versionada como workflow, sugerindo deploy manual ou via integração externa não capturada no repositório (provedor estático com integração direta ao GitHub, por exemplo).

11.12 Runtime e Empacotador

11.12.1 Node.js

Embora **não seja runtime da aplicação em produção** (que executa exclusivamente no navegador do usuário), o **Node.js** é o runtime essencial para o **tooling de desenvolvimento**: Vite, ESLint e os scripts NPM dependem dele. A aplicação é compatível com Node.js 18+ (versão mínima exigida pelo Vite 7).

11.12.2 Gerenciador de Pacotes

O `package-lock.json` presente no repositório indica o **npm** como gerenciador de pacotes padrão. O lockfile garante reprodutibilidade da árvore de dependências entre máquinas e ambientes (desenvolvimento, CI, produção). O histórico revela uma passagem pelo **Bun** (commit `4c13f88` intitulado simplesmente "bun"), aparentemente experimental e não consolidada — o repositório atual está alinhado ao npm.

11.13 Estrutura de Diretórios do Projeto

A organização de pastas reflete a separação por responsabilidade:

```

Treinamento/
├─ public/                # Assets estáticos (favicon, imagens)
├─ src/                   # Código-fonte da aplicação
│  ├─ assets/             # Assets versionados (imagens, fontes)
│  ├─ components/         # Componentes reutilizáveis
│  │  ├─ AIChat.jsx
│  │  ├─ Badges.jsx
│  │  ├─ Layout.jsx
│  │  ├─ ProtectedRoute.jsx
│  │  ├─ AdminRoute.jsx
│  │  ├─ MonitorRoute.jsx
│  │  └─ RecoveryRedirectHandler.jsx
│  ├─ contexts/           # Contextos React
│  │  └─ AuthContext.jsx
│  ├─ lib/                # Bibliotecas internas (utilitários e clientes)
│  │  ├─ supabase.js
│  │  ├─ gemini.js
│  │  ├─ badges.js
│  │  └─ accessLevels.js
│  ├─ pages/              # Páginas (rotas)
│  │  ├─ Dashboard.jsx
│  │  ├─ DisciplineDetail.jsx
│  │  ├─ Quiz.jsx
│  │  ├─ Forum.jsx
│  │  ├─ ForumPost.jsx
│  │  ├─ MyDoubts.jsx
│  │  ├─ Login.jsx
│  │  ├─ ForgotPassword.jsx
│  │  ├─ ResetPassword.jsx
│  │  ├─ admin/           # Páginas administrativas
│  │  │  ├─ AdminDisciplines.jsx
│  │  │  ├─ AdminDisciplineEdit.jsx
│  │  │  ├─ AdminUsers.jsx
│  │  │  ├─ AdminMonitors.jsx
│  │  │  └─ AdminReports.jsx
│  │  └─ monitor/         # Páginas do monitor
│  │     ├─ MonitorStudents.jsx
│  │     ├─ MonitorStudentDetail.jsx
│  │     ├─ MonitorDoubts.jsx
│  │     └─ MonitorDoubtDetail.jsx
│  ├─ App.jsx             # Componente raiz
│  ├─ main.jsx            # Entry point
│  └─ index.css           # Estilos globais
├─ supabase/              # Esquema e migrações do banco
│  ├─ schema.sql
│  ├─ migration_*.sql     # ~15 migrações nomeadas
│  └─ sql_*.sql          # Scripts operacionais datados
├─ generate_sql.py        # Gerador de SQL (descartável)
├─ package.json
├─ package-lock.json
├─ vite.config.js
└─ eslint.config.js

```

```
|— .env.example
|— README.md
|— RELATORIO_INSPECAO_TECNICA.md      # 1ª edição
|— RELATORIO_INSPECAO_TECNICA_II.md   # 2ª edição (este documento)
```

11.14 Matriz Consolidada de Versões

Para facilitar referência rápida, a tabela a seguir consolida **todas as versões** declaradas em `package.json`:

Pacote	Versão	Tipo	Função
react	^19.2.4	dependency	Framework UI
react-dom	^19.2.4	dependency	Renderização DOM
react-router-dom	^7.13.0	dependency	Roteamento SPA
react-icons	^5.5.0	dependency	Iconografia
@supabase/supabase-js	^2.95.3	dependency	Cliente do Supabase
@google/generative-ai	^0.24.1	dependency	Cliente do Gemini
xlsx	^0.18.5	dependency	Manipulação de planilhas
vite	^7.3.1	devDependency	Bundler/Dev server
@vitejs/plugin-react	^5.1.4	devDependency	Plugin React para Vite
eslint	^9.39.2	devDependency	Linter
@eslint/js	^9.39.2	devDependency	Configs base do ESLint
eslint-plugin-react-hooks	^7.0.1	devDependency	Lint de Rules of Hooks
eslint-plugin-react-refresh	^0.4.26	devDependency	Lint para Fast Refresh
globals	^16.5.0	devDependency	Identificadores globais para ESLint
@types/react	^19.2.14	devDependency	Tipagens (IntelliSense)
@types/react-dom	^19.2.3	devDependency	Tipagens (IntelliSense)

Total: 7 dependências de produção e 9 dependências de desenvolvimento — uma árvore **enxuta** quando comparada a projetos do mesmo porte, o que **reduz a superfície de vulnerabilidades transitivas** e o custo de manutenção. Esta é uma característica positiva já registrada na primeira edição.

11.15 Características Notáveis Ausentes (Considerações Arquiteturais)

Para que o inventário seja **completo** também naquilo que **não está presente**, registra-se a seguir o conjunto de tecnologias **frequentes em projetos similares e ausentes nesta plataforma** — algumas por escolha consciente, outras por dívida técnica:

- **TypeScript** — ausente; recomendado em todas as edições deste relatório;
- **Framework de testes (Vitest, Jest, Testing Library)** — ausente; cobertura de 0% (ponto crítico nº 1);
- **State manager externo (Redux, Zustand, Jotai, MobX)** — ausente, por escolha; o `useState` / `useContext` cobre as necessidades atuais;
- **Biblioteca de data-fetching (TanStack Query / SWR)** — ausente; o cliente Supabase é utilizado diretamente, sem cache transversal;
- **Pré-processador CSS (Sass, Less)** — ausente; CSS puro é suficiente;
- **Framework CSS-in-JS (styled-components, emotion)** — ausente, por escolha;
- **Framework de componentes (Material UI, Chakra, Mantine, shadcn/ui)** — ausente; a UI é construída sob medida;
- **Backend proxy / API Gateway próprio** — ausente; a aplicação é frontend-only com BaaS;
- **Edge Functions / serverless** — disponíveis no Supabase, mas **não utilizadas**;
- **Realtime / WebSocket** — disponível no Supabase, mas **não utilizado** (polling de 30s é a solução atual);
- **Internacionalização (i18n)** — ausente; a aplicação é monoidioma (português);
- **Service Worker / PWA** — ausente;
- **Sentry / monitoramento de erros em produção** — ausente;
- **Analytics** — ausente;
- **Storybook ou ferramenta de catálogo de componentes** — ausente.

Cada ausência tem suas próprias implicações; algumas são oportunidades de evolução, outras são decisões legítimas de escopo enxuto. O presente inventário registra todas para que decisões futuras possam ser tomadas com **visibilidade plena** do que está em jogo.

11.16 Conclusão do Capítulo

O stack tecnológico do Capacita Portos é **moderno, enxuto e coerente** — alinhado às melhores práticas atuais do ecossistema JavaScript para SPAs com BaaS. As decisões fundamentais (React 19, Vite 7, Supabase, Gemini) são **bem fundamentadas tecnicamente** e oferecem uma base sólida para a evolução continuada da plataforma e para a futura construção do Capacita Portos Interno descrita no capítulo 10.

A árvore de dependências é **deliberadamente pequena**, característica que merece preservação à medida que novas funcionalidades forem incorporadas. A introdução de cada nova dependência deve ser **deliberada e justificada**, com avaliação prévia de seu **custo de manutenção, peso no bundle e risco de segurança**. O período coberto por esta edição respeitou esse princípio — apenas a biblioteca `xlsx` foi adicionada, com motivação clara e escopo de uso restrito.

Recomenda-se, finalmente, que este capítulo seja **mantido atualizado** em cada edição futura deste relatório, servindo como **registro de verdade sobre o estado tecnológico** da plataforma ao longo do tempo. Mudanças relevantes — adições, remoções, atualizações majoras de versão — devem ser refletidas aqui, viabilizando a comparação histórica do stack edição após edição.

APÊNDICE A — DETALHAMENTO DOS COMMITS DO PERÍODO

Campo	Valor
Commit	0894ccf5
Data/hora	06/04/2026 09:45 (-03:00)
Mensagem	Add SQL scripts for user email updates and password reset enforcement
Arquivos	4 (generate_sql.py , 3 scripts SQL)
Linhas	+1.602 / -37
Capítulo deste relatório	3

Campo	Valor
Commit	25effa47
Data/hora	13/04/2026 10:32 (-03:00)
Mensagem	feat: add access levels and database migration for user roles
Arquivos	7 (6 de código + o arquivo da 1ª edição do relatório)
Linhas	+1.854 / -4 (sendo +249 / -4 de código de aplicação)
Capítulo deste relatório	4

Campo	Valor
Commit	ba5232f5
Data/hora	30/04/2026 11:38 (-03:00)
Mensagem	feat: add Excel export functionality for student reports and update dependencies
Arquivos	4 (3 de código + package-lock.json)
Linhas	+254 / -28
Capítulo deste relatório	5

Campo	Valor
Commit	15fe2250
Data/hora	19/05/2026 17:32 (-03:00)
Mensagem	feat: add migration to allow admin to delete forum posts and replies
Arquivos	1 (migration_forum_admin_delete.sql)
Linhas	+24 / -0
Capítulo deste relatório	6

Campo	Valor
Commit	e081751a
Data/hora	19/05/2026 17:36 (-03:00)
Mensagem	feat: add functionality to manage quiz options with add and remove buttons
Arquivos	2 (AdminDisciplineEdit.jsx , AdminDisciplineEdit.css)
Linhas	+71 / -0
Capítulo deste relatório	7

APÊNDICE B — ÍNDICE DE ARQUIVOS AFETADOS NO PERÍODO

Arquivo	Entrega(s)	Status	Capítulo
generate_sql.py	0894ccf	Novo	3
supabase/ sql_forcar_troca_senha_novos_alunos_2026_03_31.sql	0894ccf	Novo	3
supabase/sql_inserir_novos_alunos_2026_03_30.sql	0894ccf	Novo	3
supabase/sql_trocar_emails_2026_03_30.sql	0894ccf	Modificado	3
src/lib/accessLevels.js	25effa4	Novo	4
supabase/migration_access_levels.sql	25effa4	Novo	4
src/contexts/AuthContext.jsx	25effa4	Modificado	4
src/pages/DisciplineDetail.jsx	25effa4	Modificado	4
src/pages/DisciplineDetail.css	25effa4	Modificado	4
src/pages/admin/AdminUsers.jsx	25effa4	Modificado	4
package.json	ba5232f	Modificado	5
package-lock.json	ba5232f	Modificado (auto)	5
src/pages/admin/AdminReports.jsx	ba5232f	Modificado	5
src/pages/admin/AdminReports.css	ba5232f	Modificado	5
supabase/migration_forum_admin_delete.sql	15fe225	Novo	6
src/pages/admin/AdminDisciplineEdit.jsx	e081751	Modificado	7
src/pages/admin/AdminDisciplineEdit.css	e081751	Modificado	7

APÊNDICE C — GLOSSÁRIO

Termo	Definição
RLS (Row Level Security)	Mecanismo do PostgreSQL que aplica regras de acesso linha a linha, no nível do banco de dados, inviolável a partir do cliente.
RPC (Remote Procedure Call)	No contexto do Supabase, função do banco de dados invocável pelo frontend via <code>supabase.rpc(...)</code> .
SECURITY DEFINER	Modificador de função PostgreSQL que faz a função executar com os privilégios de quem a criou, e não de quem a chama.
Upsert	Operação que insere um registro novo ou, se já existir, atualiza o existente (<code>INSERT ... ON CONFLICT DO UPDATE</code>).
Idempotência	Propriedade de uma operação que produz o mesmo resultado independentemente de quantas vezes é executada.
PII (Personally Identifiable Information)	Informação pessoal identificável — dados que permitem identificar um indivíduo (e-mail, nome etc.).
LGPD	Lei Geral de Proteção de Dados (Lei nº 13.709/2018).
Atualização otimista (Optimistic Update)	Padrão de UX em que a interface é atualizada imediatamente, antes da confirmação do servidor, com rollback em caso de falha.
Fail-safe default	Princípio de design pelo qual, em caso de erro ou dado inválido, o sistema recai no estado mais seguro/restritivo.
Gating de conteúdo	Restrição de acesso a determinado conteúdo conforme uma condição (aqui, o nível de acesso do usuário).
Bounded Context	Conceito do Domain-Driven Design: fronteira lógica dentro da qual um modelo de domínio é coeso e consistente.
Complexidade Ciclomática (CC)	Métrica que conta o número de caminhos lineares independentes através de um trecho de código.
Scaffold (andaime)	Estrutura preliminar de uma funcionalidade, pronta para receber conteúdo, mas ainda sem entregá-lo.
Soft delete (exclusão lógica)	Marcar um registro como removido (ex.: coluna <code>deleted_at</code>) sem apagá-lo fisicamente, preservando o histórico.

FIM DO RELATÓRIO — SEGUNDA EDIÇÃO

Documento preparado: Análise Técnica de Evolução Período analisado: 6 de abril de 2026 a 19 de maio de 2026 Data de emissão: 19 de maio de 2026 Documento de referência: Relatório de Inspeção Técnica Exaustivo (1ª edição) Classificação: Confidencial — Uso Técnico Interno